

DOKUMENTASI PENGUJIAN SOFTWARE QUALITY

PADA APLIKASI LOCALESPRO

Di susun untuk memenuhi UAS Software Quality

Dosen Pengampu Deni Suprihadi, S.T, M.Kom., MCE



Disusun Oleh :

Anang Ismail	20231310056
Ilham Ramdan	20231310061
Shihad	20231310074
Dylan Malik Herlambang	20231310059

PRODI TEKNIK INFORMATIKA
FASKULTAS ILMU KOMPUTER DAN SISTEM INFORMASI
UNIVERSITAS KEBANGSAAN REPUBLIK INDONESIA

2026

KATA PENGANTAR

Puji syukur kami panjatkan kehadirat Allah SWT atas segala rahmat dan karunia-Nya sehingga kami dapat menyelesaikan laporan dokumentasi ini dengan judul "Dokumentasi Pengujian Software Quality Aplikasi LocalesPro" sebagai salah satu tugas Ujian Akhir Semester (UAS) pada mata kuliah Software Quality.

Makalah ini disusun dengan tujuan untuk mendokumentasikan cara pengujian aplikasi menggunakan 3 pengujian yaitu Whitebox, Blackbox, dan Greybox testing yang diperlukan dalam pengembangan aplikasi LocalesPro. Dalam penyusunannya, kami berusaha menyajikan pembahasan yang sistematis dan komprehensif sesuai dengan konsep-konsep Software Quality yang telah dipelajari.

Kami menyadari bahwa makalah ini masih jauh dari sempurna. Oleh karena itu, kami mengucapkan terima kasih kepada Bapak Deni Supriyadi, S.T, M.Kom., MCE. selaku dosen pengampu mata kuliah Software Quality atas bimbingan dan arahannya selama perkuliahan berlangsung. Kami juga mengucapkan terima kasih kepada semua pihak yang telah memberikan dukungan dan bantuan dalam penyusunan makalah ini.

Kami berharap makalah ini dapat bermanfaat dan memberikan wawasan mengenai pentingnya Dokumentasi Pengujian Software Quality Aplikasi LocalesPro. Kami juga terbuka menerima kritik dan saran yang membangun untuk perbaikan di masa mendatang.

Bandung, 21 Juni 2026

DAFTAR ISI

KATA PENGANTAR.....	2
DAFTAR ISI	3
BAB I PENDAHULUAN	7
1.1 Latar Belakang	7
1.2 Rumusan Masalah	8
1.3 Tujuan Penelitian	9
1.4 Batasan Masalah	10
BAB II LANDASAN TEORI	12
2.1 Software Quality Assurance	12
2.2 Testing	13
2.3 Black Box Testing	13
2.3.1 Equivalence Partitioning.....	14
2.3.2 Boundory Value Analysis.....	14
2.3.3 Decision Table Testing.....	15
2.3.4 Cause-Effect Relationship Testing.....	16
2.3.5 Robustness Testing.....	17
2.4 White Box Testing	17
2.4.1 Desk Checking	18
2.4.2 Code Walkthrough	18
2.4.3 Control Flow Testing	18
2.4.4 Data Flow Testing	19
2.4.5 Basic Path Testing.....	19
2.5 Grey Box Testing.....	20
2.5.1 Orthogonal Array Testing	21
2.5.2 Matrix Testing.....	21
2.5.3 Refression Testing.....	22
2.5.4 Pattern Testing	22
2.6 GitHub sebagai Media Komunikasi dan Dokumentasi Pengujian	23
2.6.1 GitHub sebagai Media Komunikasi.....	23
2.6.2 GitHub sebagai penyimpanan data pengujian	24

2.6.3 Github Sebagai Dokumentasi Bug Fixing	24
2.6.4 Keuntungan menggunakan Github untuk SQA	25
BAB III OBJEK DAN LINGKUNGAN PENGUJIAN.....	26
3.1 Deskripsi Aplikasi	26
3.2 Fitur yang diuji	26
3.3 Alur bisnis fitur bahan baku	26
3.4 Teknologi	27
3.5 Source Code yang relevan	27
3.6 Struktur Repository GitHub	28
3.7 Data uji global	28
3.8 Kriteria keberhasilan umum	29
BAB IV PERANCANGAN PENGUJIAN	30
4.1 Pembuatan Repository Github.....	30
4.2 Penambahan Collaborator	31
4.3 Pengelolaan File dan Commit Proyek	34
4.4 Komunikasi antara Develover dan SQA Tester	36
4.5 Strategi Pengujian.....	37
4.6 Lingkungan pnegujian	38
4.6.1 Perangkat keras	38
4.6.2 Perangkat lunak.....	38
4.6.3 Aplikasi yang diuji	39
4.7 Data Uji	39
4.7.1 Data Produk dan Transaksi	39
4.7.2 Data Resep produk.....	39
4.7.3 Expected stok setelah transaksi.....	39
4.7.4 Kondisi Awal pengujian	40
4.8 Rencana pelaksanaan pengujian	40
4.8.1 Tahapan pengujian	40
4.9 Kriteria Keberhasilan pengujian.....	40
4.9.1 Kriteria Keberhasilan Black Box	40
4.9.2 Kriteria keberhasilan White box	41
4.9.3 Kriteria Keberhasilan Grey box	41

4.10 Format Pencatatan hasil.....	42
4.10.1 Format hasil blalck box.....	42
4.10.2 Format hasil white box	42
4.10.3 Format Gasil Grey Box.....	42
4.11 Traceability Matrix	43
4.12 Dokumentasi dan pelaporan	43
BAB V HASIL BLACK BOX TESTING.....	44
5.1 Ringkasan Balck Box Testing.....	44
5.2 Metode yang Digunakan.....	45
5.2.1 Equivalence Partitioning.....	45
5.2.2 Boundary Value Analysis.....	48
5.2.3 Decision Table Testing.....	52
5.2.4 Cause-Effect Relationship Testing.....	55
5.2.5 Robustness Testing.....	58
5.3 Ringkasan Hasil Black Box.....	62
5.4 Temuan Black Box	63
BAB VI HASIL WHITE BOX TESTING	65
6.1 Ringkasan White Box testing	65
6.2 Metode yang Digunakan.....	65
6.2.1 Desk Checking.....	65
6.2.2 Code Walkthrough	69
6.2.3 Control Flow Testing	71
6.2.4 Data Flow Testing :.....	73
6.2.5 Basic Path Testing.....	74
6.3 Cyclomatic Complexity.....	75
6.4 Ringkasan Hasil White Box	76
6.5 Temuan White Box	76
6.6 Kesimpulan.....	77
BAB VII.....	78
7.1 Ringkasan Grey Box Testing.....	78
7.2 Metode yang digunakan	78
7.2.1 Orthogonal Array Testing	78

7.2.2 Matrix Testing.....	82
7.2.3 Regression Testing	86
7.2.4 Pattern Testing	90
7.3 Ringkasan Hasil Grey Box.....	95
7.4 Query Verifikasi.....	95
7.5 Temukan Grey Box.....	96
7.6 Kesimpulan.....	97
BAB VIII ANALISIS PERBANDINGAN PENGUJIAN	98
8.1 Perbandingan Black Box, White Box dan Grey Box	98
8.2 Hubungan Antar Pengujian	100
8.2.1 Black Box → White Box	100
8.2.2 White Box → Grey Box	100
8.2.3 Grey Box → Black Box.....	101
8.3 Analisis Kualitas Fitur	101
8.3.1 Validasi Produk	101
8.3.2 Validasi Resep.....	102
8.3.3 Validasi Stok	102
8.3.4 Transaksi Database	103
8.3.5 Pengurangan Stok	103
8.3.6 Mutasi Stok.....	104
8.3.7 Void Transaksi.....	104
8.4 Coverage Pengujian.....	104
8.5 Temuan Lintas Pengujian	105
BAB IX KESIMPULAN DAN SARAN	106
9.1 Kesimpulan.....	106
9.2 Saran.....	107
DAFTAR PUSTAKA.....	109

BAB I PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi informasi telah mendorong berbagai sektor usaha untuk memanfaatkan sistem informasi dalam meningkatkan efektivitas, efisiensi, dan akurasi pengelolaan operasional bisnis. Salah satu bentuk penerapan teknologi tersebut adalah penggunaan aplikasi Point of Sale (POS) yang terintegrasi dengan sistem manajemen persediaan (*inventory management*) untuk mendukung aktivitas transaksi penjualan, pengelolaan produk, serta pengendalian stok bahan baku.

LocalesPro merupakan aplikasi Point of Sale (POS) yang dikembangkan untuk membantu operasional usaha kafe atau penjualan minuman. Aplikasi ini memiliki beberapa fitur utama, seperti pengelolaan bahan baku, pengelolaan produk, pembuatan resep produk, transaksi penjualan, pengelolaan stok, mutasi stok, serta laporan transaksi. Seluruh fitur tersebut saling terhubung sehingga setiap aktivitas yang dilakukan pada satu modul akan memengaruhi modul lainnya.

Dalam proses pengembangan perangkat lunak, kerja sama tim menjadi salah satu faktor penting yang menentukan keberhasilan suatu proyek. Oleh karena itu, diperlukan sebuah platform yang dapat memfasilitasi kolaborasi, pengelolaan kode sumber, dokumentasi pekerjaan, serta pengendalian perubahan sistem secara terstruktur. GitHub digunakan sebagai media kolaborasi dalam pengembangan aplikasi LocalesPro, mulai dari pembuatan repository, penambahan collaborator, pengaturan hak akses anggota, pengelolaan file proyek, pembuatan issue, hingga proses perbaikan (*bug fixing*) yang ditemukan selama pengembangan sistem.

Selain proses pengembangan, kualitas perangkat lunak juga menjadi aspek yang sangat penting untuk diperhatikan. Kesalahan yang terjadi pada sistem dapat menyebabkan berbagai permasalahan, seperti ketidaksesuaian data stok, kesalahan perhitungan transaksi, pencatatan mutasi yang tidak akurat, serta ketidaksesuaian laporan yang dihasilkan. Permasalahan tersebut dapat mengganggu aktivitas operasional dan menurunkan kepercayaan pengguna terhadap sistem yang digunakan.

Salah satu fitur yang memiliki peranan penting pada aplikasi LocalesPro adalah fitur bahan baku. Setiap transaksi penjualan yang dilakukan melalui POS akan secara otomatis mengurangi jumlah stok bahan baku berdasarkan resep yang telah ditentukan sebelumnya. Oleh karena itu, diperlukan mekanisme pengujian yang mampu memastikan bahwa proses validasi produk, pengurangan stok, pencatatan mutasi, serta proses pembatalan (*void transaction*) berjalan dengan baik dan sesuai dengan kebutuhan sistem.

Untuk menjamin kualitas perangkat lunak, diterapkan konsep Software Quality Assurance (SQA) melalui beberapa metode pengujian, yaitu Black Box Testing, White Box Testing, dan Grey Box Testing. Black Box Testing digunakan untuk menguji fungsi sistem berdasarkan masukan dan keluaran yang dihasilkan tanpa melihat struktur internal program. White Box Testing digunakan untuk menganalisis logika, alur program, dan struktur kode yang digunakan. Sementara itu, Grey Box Testing digunakan untuk mengombinasikan pengujian fungsional dengan verifikasi terhadap struktur internal sistem, seperti database dan proses yang berjalan di belakang layar.

Melalui penerapan ketiga metode pengujian tersebut, diharapkan kualitas aplikasi LocalesPro dapat ditingkatkan serta mampu menghasilkan sistem yang akurat, konsisten, dan andal. Selain itu, dokumentasi pengembangan proyek menggunakan GitHub diharapkan dapat memberikan gambaran mengenai proses pengelolaan proyek perangkat lunak yang dilakukan secara kolaboratif, terstruktur, dan terdokumentasi dengan baik, mulai dari tahap perencanaan hingga tahap pengujian dan evaluasi sistem.

1.2 Rumusan Masalah

Berdasarkan latar belakang yang telah diuraikan, maka rumusan masalah pada penelitian ini adalah sebagai berikut:

1. Bagaimana proses pengembangan aplikasi LocalesPro dilakukan secara kolaboratif menggunakan GitHub?
2. Bagaimana proses pembuatan dan pengelolaan repository GitHub sebagai pusat pengembangan aplikasi LocalesPro?

3. Bagaimana proses penambahan collaborator dan pengaturan hak akses anggota tim pada GitHub?
4. Bagaimana proses pengelolaan file, dokumentasi, dan penggunaan GitHub Issue dalam mendukung pengembangan aplikasi?
5. Bagaimana proses identifikasi dan perbaikan (*bug fixing*) selama pengembangan aplikasi LocalesPro?
6. Bagaimana implementasi fitur bahan baku, produk, resep produk, POS, stok, mutasi stok, laporan transaksi, dan *void transaction* pada aplikasi LocalesPro?
7. Apakah fitur produk dan resep bahan baku pada aplikasi LocalesPro berjalan sesuai dengan kebutuhan sistem?
8. Apakah transaksi POS berhasil mengurangi stok bahan baku sesuai dengan resep yang telah ditentukan?
9. Apakah transaksi yang gagal diproses tidak mengubah data stok bahan baku?
10. Bagaimana penerapan metode Black Box Testing, White Box Testing, dan Grey Box Testing dalam menjamin kualitas aplikasi LocalesPro?
11. Bagaimana kualitas aplikasi setelah dilakukan proses pengujian dan evaluasi sistem?

1.3 Tujuan Penelitian

Berdasarkan rumusan masalah yang telah disusun, maka tujuan penelitian ini adalah sebagai berikut:

1. Menjelaskan proses pengembangan aplikasi LocalesPro secara kolaboratif menggunakan GitHub.
2. Menjelaskan proses pembuatan dan pengelolaan repository GitHub sebagai media pengembangan proyek.
3. Menjelaskan proses penambahan collaborator dan pengaturan hak akses anggota tim.

4. Menjelaskan proses pengelolaan file, dokumentasi proyek, dan penggunaan GitHub Issue.
5. Menjelaskan proses identifikasi dan perbaikan bug yang ditemukan selama pengembangan aplikasi.
6. Mengimplementasikan fitur bahan baku, produk, resep produk, POS, stok, mutasi stok, laporan transaksi, dan void transaction pada aplikasi LocalesPro.
7. Menguji kesesuaian fitur produk dan resep bahan baku dengan kebutuhan sistem.
8. Memastikan transaksi POS dapat mengurangi stok bahan baku sesuai resep yang telah ditentukan.
9. Memastikan transaksi yang gagal tidak mengubah data stok bahan baku.
10. Menerapkan metode Black Box Testing, White Box Testing, dan Grey Box Testing untuk menjamin kualitas perangkat lunak.
11. Mengevaluasi kualitas aplikasi setelah seluruh proses pengembangan dan pengujian dilakukan.

1.4 Batasan Masalah

Agar pembahasan lebih terarah dan tidak menyimpang dari tujuan penelitian, maka batasan masalah pada laporan ini adalah sebagai berikut:

1. Pengembangan sistem dilakukan menggunakan platform GitHub sebagai media kolaborasi tim.
2. Pengelolaan GitHub yang dibahas meliputi pembuatan repository, penambahan collaborator, pengaturan hak akses, pengelolaan file, pembuatan issue, dan dokumentasi bug fixing.
3. Aplikasi yang dibahas adalah LocalesPro yang berfungsi sebagai sistem Point of Sale (POS).
4. Fitur yang diuji meliputi bahan baku, produk, resep produk, POS, stok, mutasi stok, laporan transaksi, dan void transaction.
5. Pengujian sistem dilakukan menggunakan metode Black Box Testing, White Box Testing, dan Grey Box Testing.

6. Pengujian tidak membahas aspek keamanan sistem secara mendalam, seperti enkripsi data, keamanan token, dan penetration testing.
7. Pengujian tidak membahas performa server dalam skala penggunaan yang sangat besar.
8. Pengujian tidak membahas desain antarmuka pengguna (User Interface) secara detail.
9. Database yang digunakan mengacu pada struktur database yang diterapkan pada aplikasi LocalesPro.

BAB II LANDASAN TEORI

2.1 Software Quality Assurance

Software Quality Assurance (SQA) merupakan serangkaian aktivitas yang dilakukan secara sistematis untuk memastikan bahwa perangkat lunak yang dikembangkan telah memenuhi kebutuhan, standar kualitas, dan spesifikasi yang telah ditetapkan. SQA tidak hanya berfokus pada hasil akhir perangkat lunak, tetapi juga mencakup seluruh proses pengembangan mulai dari tahap perencanaan, implementasi, pengujian, evaluasi, hingga perbaikan kualitas secara berkelanjutan.

Tujuan utama SQA adalah meminimalkan kesalahan (error), mencegah terjadinya cacat (defect), serta meningkatkan keandalan, keamanan, dan kualitas perangkat lunak sebelum digunakan oleh pengguna. Dengan menerapkan SQA, proses pengembangan perangkat lunak menjadi lebih terstruktur sehingga dapat menghasilkan sistem yang sesuai dengan kebutuhan pengguna.

Secara umum, Software Quality Assurance mencakup beberapa aktivitas utama, yaitu:

1. Perencanaan kualitas perangkat lunak.
2. Pengelolaan proses pengembangan perangkat lunak.
3. Pelaksanaan pengujian perangkat lunak.
4. Evaluasi hasil pengujian.
5. Perbaikan kesalahan (bug fixing).
6. Dokumentasi seluruh proses pengembangan dan pengujian.

Pada penelitian ini, konsep SQA diterapkan pada aplikasi LocalesPro untuk memastikan seluruh fitur, khususnya fitur bahan baku, resep produk, transaksi POS, stok, mutasi stok, dan pembatalan transaksi dapat berjalan dengan baik sesuai kebutuhan sistem.

2.2 Testing

Testing atau pengujian perangkat lunak merupakan proses evaluasi terhadap suatu sistem untuk menemukan kesalahan, kekurangan, atau ketidaksesuaian antara hasil yang dihasilkan dengan kebutuhan yang telah ditetapkan.

Tujuan utama testing adalah memastikan bahwa perangkat lunak dapat bekerja sesuai dengan spesifikasi, memenuhi kebutuhan pengguna, serta meminimalkan kemungkinan terjadinya kesalahan saat sistem digunakan.

Pengujian dilakukan dengan memberikan berbagai kondisi masukan (*input*) dan mengamati keluaran (*output*) yang dihasilkan oleh sistem. Apabila keluaran yang dihasilkan sesuai dengan yang diharapkan, maka sistem dapat dinyatakan berjalan dengan baik.

Beberapa tujuan dilakukannya testing antara lain:

1. Menemukan kesalahan yang terdapat pada sistem.
2. Memastikan fungsi sistem berjalan sesuai kebutuhan.
3. Memastikan integrasi antar modul berjalan dengan baik.
4. Meningkatkan kualitas perangkat lunak.
5. Mengurangi risiko kegagalan sistem saat digunakan.

Pada penelitian ini, pengujian dilakukan menggunakan tiga pendekatan utama, yaitu Black Box Testing, White Box Testing, dan Grey Box Testing.

2.3 Black Box Testing

Black Box Testing merupakan metode pengujian perangkat lunak yang berfokus pada pengujian fungsi sistem berdasarkan masukan (*input*) dan keluaran (*output*) tanpa melihat struktur internal kode program.

Pada metode ini, penguji hanya memeriksa apakah sistem menghasilkan keluaran yang sesuai dengan kebutuhan yang telah ditentukan. Penguji tidak perlu mengetahui bagaimana kode program dibuat atau bagaimana proses internal sistem bekerja.

Tujuan Black Box Testing adalah memastikan bahwa setiap fungsi sistem dapat berjalan sesuai dengan spesifikasi yang telah dirancang.

Karakteristik Black Box Testing adalah sebagai berikut:

1. Berfokus pada fungsi sistem.
2. Menguji input dan output.
3. Tidak memerlukan akses terhadap source code.
4. Menggunakan skenario pengujian berdasarkan kebutuhan pengguna.

Pada penelitian ini digunakan beberapa metode Black Box Testing, yaitu:

2.3.1 Equivalence Partitioning

Equivalence Partitioning merupakan metode pengujian yang membagi data masukan ke dalam kelompok data valid dan tidak valid. Pengujian dilakukan dengan memilih perwakilan dari setiap kelompok data sehingga jumlah test case dapat dikurangi tanpa mengurangi efektivitas pengujian.

Contoh penerapan pada LocalesPro:

- Partisi valid: produk dengan resep lengkap (Kopisusu dengan Sedotan 1 pcs, Keju 100 gr, Sirup Gula Aren 10 ml), stok mencukupi (Sedotan 10 pcs, Keju 1.000 gr, Sirup 1.500 ml), qty transaksi positif (3 porsi), pembayaran cukup (Rp 50.000 untuk total Rp 30.000).
- Partisi tidak valid: produk tanpa resep, stok kurang (Sedotan 2 pcs untuk kebutuhan 3 pcs), qty nol atau negatif, pembayaran kurang dari total tagihan.

2.3.2 Boundary Value Analysis

Boundary Value Analysis merupakan metode pengujian yang berfokus pada nilai-nilai batas (boundary) dari suatu kelompok data input. Metode ini didasarkan pada asumsi bahwa kesalahan cenderung terjadi pada nilai batas daripada nilai tengah.

Pengujian dilakukan dengan memilih nilai-nilai yang berada tepat pada batas, tepat di bawah batas, dan tepat di atas batas dari suatu range input yang valid.

Contoh penerapan pada LocalesPro:

Batas	Nilai Uji	Keterangan
Stok bahan baku	0	Stok habis (tidak valid)
Stok bahan baku	1	Stok minimal (valid)
Qty transaksi	0	Tidak pembelian (tidak valid)
Qty transaksi	1	Pembelian minimal (valid)
Pembayaran	Rp 29.999	Kurang dari harga (tidak valid)
Pembayaran	Rp 30.000	Pas dengan harga (valid)
Pembayaran	Rp 30.001	Lebih dari harga (valid)
Alasan void	2 karakter	Kurang dari batas minimum (tidak valid)
Alasan void	3 karakter	Batas minimum valid

2.3.3 Decision Table Testing

Decision Table Testing merupakan metode pengujian yang digunakan untuk menguji berbagai kombinasi kondisi dan keputusan yang mungkin terjadi pada sistem. Metode ini membantu memastikan bahwa seluruh kombinasi kondisi telah diuji secara menyeluruh.

Contoh Penerapan pada LocalesPro:

Kondisi	Kasus 1	Kasus 2	Kasus 3	Kasus 4
Produk aktif	Ya	Ya	Ya	Tidak

Stok cukup	Ya	Ya	Tidak	Ya
Qty valid	Ya	Tidak	Ya	Ya
Uang cukup	Ya	Ya	ya	Ya
Hasil	Transaksi Berhasil	Gagal: Qty tidak valid	Gagal: Stok tidak cukup	Gagal: Produk tidak aktif

2.3.4 Cause-Effect Relationship Testing

Cause-Effect Relationship Testing merupakan metode pengujian yang digunakan untuk mengidentifikasi hubungan antara penyebab (cause) dan akibat (effect) yang dihasilkan oleh sistem.

Sebagai contoh, transaksi yang berhasil akan menyebabkan stok berkurang, sedangkan transaksi yang dibatalkan akan menyebabkan stok kembali seperti semula.

Contoh penerapan pada LocalesPro:

Cause (Penyebab)	Effect (Akbiat)
Kasir memilih produk “Kopisusu”	Sistem menampilkan harga dan resep bahan baku
Kasir Input qty = 3	Sistem menghitung kebutuhan bahan: Sedotan 3 pcs, Keju 300 gr, Sirup 30 ml
Sistem validasi stok = cukup	Transaksi dapat diproses
Pembayaran Rp 50.00 untuk total Rp 30.000	Sistem menghitung kembalian Rp 20.000
Transaksi berhasil disimpan	Stok bahan baku berkurang sesuai resep x qty
Transaksi di-void	Stok bahan baku dikembalikan ke jumlah semula

2.3.5 Robustness Testing

Robustness Testing merupakan metode pengujian yang digunakan untuk menguji kemampuan sistem dalam menangani kondisi ekstrem atau masukan yang tidak normal. Pengujian ini bertujuan untuk mengetahui apakah sistem tetap stabil ketika menerima data yang tidak sesuai.

Contoh penerapan pada LocalesPro:

- Input qty transaksi berupa karakter/huruf.
- Input harga produk berupa nilai negatif.
- Input stok bahan baku berupa nilai desimal yang tidak sesuai satuan.
- Membatalkan transaksi yang sudah dibatalkan sebelumnya (double void).
- Mengakses fitur POS dengan akun yang tidak memiliki hak akses kasir.
- Klik tombol konfirmasi pembayaran berulang kali untuk mencegah transaksi ganda.

2.4 White Box Testing

White Box Testing merupakan metode pengujian yang berfokus pada struktur internal perangkat lunak, source code, percabangan program, aliran data, serta jalur eksekusi yang digunakan oleh sistem.

Berbeda dengan Black Box Testing, metode ini mengharuskan penguji memahami logika dan struktur kode yang digunakan dalam pengembangan sistem.

Tujuan White Box Testing adalah memastikan bahwa seluruh jalur logika program dapat berjalan dengan baik dan meminimalkan kemungkinan terjadinya kesalahan pada kode program.

Karakteristik White Box Testing adalah sebagai berikut:

1. Berfokus pada source code.
2. Menguji struktur internal sistem.

3. Menganalisis percabangan program.
4. Menganalisis aliran data.
5. Menguji jalur eksekusi program.

Pada penelitian ini digunakan beberapa metode White Box Testing, yaitu:

2.4.1 Desk Checking

Desk Checking merupakan metode pemeriksaan kode program secara manual untuk memastikan bahwa logika program dan nilai variabel berjalan sesuai dengan yang diharapkan.

Contoh penerapan pada LocalesPro:

Memeriksa variabel `$stock_before`, `$stock_needed`, dan `$stock_after` pada fungsi `applyInventoryUsageForTransaction()` di file `backend/config/payment_helpers.php`. Contohnya, jika stok awal Keju = 1.000 gram, kebutuhan per produk = 100 gram, dan `qty` = 3, maka nilai `$stock_after` harus menjadi 700 gram.

2.4.2 Code Walkthrough

Code Walkthrough merupakan proses peninjauan kode program secara sistematis untuk mengidentifikasi kesalahan logika, struktur, dan implementasi kode.

Contoh penerapan pada LocalesPro: Review kode pada file `backend/api/transactions/create.php` untuk memastikan urutan eksekusi: (1) validasi produk, (2) validasi resep, (3) validasi stok, (4) simpan transaksi, (5) kurangi stok, (6) catat mutasi.

2.4.3 Control Flow Testing

Control Flow Testing merupakan metode pengujian yang digunakan untuk menganalisis alur eksekusi program berdasarkan node dan edge yang terdapat pada flow graph.

Contoh penerapan pada LocalesPro:

Menganalisis alur transaksi POS dari input produk hingga penyimpanan ke database, memetakan node (proses) dan edge (alur) untuk mengidentifikasi jalur sukses dan jalur gagal. Berdasarkan file control-flow-testing-bahan-baku.md, terdapat 19 node dan 13 edge pada alur transaksi POS.

2.4.4 Data Flow Testing

Data Flow Testing merupakan metode pengujian yang digunakan untuk menganalisis bagaimana data didefinisikan, digunakan, dan dimodifikasi selama proses eksekusi program berlangsung.

Contoh penerapan pada LocalesPro:

Menganalisis aliran data stok bahan baku dari tabel ingredients → proses validasi di payment_helpers.php → pengurangan stok → pencatatan ke tabel stock_movements. Variabel utama yang ditelusuri: \$quantity, \$requiredPerItem, \$usageByIngredient, \$available, \$required, \$delta, \$stockAfter.

2.4.5 Basic Path Testing

Basic Path Testing merupakan metode pengujian yang digunakan untuk mengidentifikasi seluruh jalur independen dalam program berdasarkan nilai Cyclomatic Complexity.

Cyclomatic Complexity ($V(G)$) merupakan metrik yang digunakan untuk mengukur kompleksitas logika program. Rumus yang digunakan:

$$V(G) = \text{jumlah predicate node} + 1$$

Atau

$$V(G) = E - N + 2P$$

Keterangan:

- $V(G)$ = Cyclomatic Complexity
- E = jumlah edge (garis alur)
- N = jumlah node (titik proses)

- P = jumlah connected component (biasanya 1 untuk program tunggal)

Contoh penerapan pada LocalesPro:

Berdasarkan file `basic-path-testing-bahan-baku.md`, pada fungsi `validateInventoryAvailabilityForCart()` di `backend/config/payment_helpers.php` terdapat 10 predicate node (percabangan if/else, loop, dan validasi). Maka:

$$V(G) = 10 + 1$$

$$V(G) = 11$$

2.5 Grey Box Testing

Grey Box Testing merupakan metode pengujian yang mengombinasikan Black Box Testing dan White Box Testing. Pada metode ini, penguji memiliki pengetahuan sebagian terhadap struktur internal sistem, namun tetap melakukan pengujian dari sudut pandang pengguna.

Grey Box Testing memungkinkan penguji untuk memverifikasi hubungan antara antarmuka pengguna, API, database, dan proses internal yang berjalan di belakang sistem.

Tujuan Grey Box Testing adalah meningkatkan efektivitas pengujian dengan mengombinasikan pengujian fungsional dan verifikasi internal sistem.

Karakteristik Grey Box Testing adalah sebagai berikut:

1. Menggabungkan Black Box Testing dan White Box Testing.
2. Penguji mengetahui sebagian struktur internal sistem.
3. Menguji integrasi antar modul.
4. Menguji hubungan antara antarmuka pengguna dan database.
5. Memudahkan identifikasi sumber kesalahan sistem.

Pada penelitian ini digunakan beberapa metode Grey Box Testing, yaitu:

2.5.1 Orthogonal Array Testing

Orthogonal Array Testing merupakan metode pengujian yang digunakan untuk menguji berbagai kombinasi faktor dan level secara efisien tanpa harus menguji seluruh kemungkinan kombinasi yang ada.

Contoh penerapan pada LocalesPro:

Faktor	Level 1	Level 2	Level 3
Produk	Aktif	Tidak aktif	-
Resep	Valid	Kosong	Jumlah bahan tidak valid
Stok	Cukup	Sama kebutuhan	Kurang
Qty	1	3	0 / tidak valid
Pembayaran	Pas total	Lebih dari total	Kurang dari total

Dengan 5 faktor dan 3 level, exhaustive testing akan menghasilkan $3^5 = 243$ kombinasi. Dengan Orthogonal Array L9, kombinasi dapat direpresentasikan dengan 9 test case utama yang tetap representatif.

2.5.2 Matrix Testing

Matrix Testing merupakan metode pengujian yang digunakan untuk menganalisis hubungan antar modul yang saling berinteraksi di dalam sistem.

Contoh penerapan pada LocalesPro:

Membuat matriks hubungan antara tabel products, product_ingredients, ingredients, transactions, transaction_items, stock_movements, dan transaction_voids untuk memastikan integritas data antar modul. Verifikasi dilakukan dengan query SQL:

```
SELECT id, name, stock_quantity, unit, branch_id
FROM ingredients
WHERE name IN ('Sedotan', 'Keju', 'Sirup Gula Aren');
```

```
SELECT id, transaction_code, total_price, payment_status,  
branch_id  
  
FROM transactions  
  
ORDER BY id DESC  
  
LIMIT 5;
```

```
SELECT ingredient_id, movement_type, direction, quantity,  
stock_before, stock_after  
  
FROM stock_movements  
  
ORDER BY id DESC  
  
LIMIT 10;
```

2.5.3 Refression Testing

Regression Testing merupakan metode pengujian yang dilakukan setelah adanya perubahan atau perbaikan sistem untuk memastikan bahwa fitur yang sebelumnya berjalan dengan baik tidak mengalami kerusakan.

Contoh penerapan pada LocalesPro:

Setelah menambahkan fitur void transaksi, dilakukan pengujian ulang pada fitur transaksi POS normal untuk memastikan fitur lama tetap berfungsi dengan baik. Test suite regression mencakup: login, tambah bahan, tambah produk, edit resep, POS transaksi berhasil, POS stok kurang, pembayaran kurang, mutasi stok, void transaksi, dan laporan transaksi.

2.5.4 Pattern Testing

Pattern Testing merupakan metode pengujian yang digunakan untuk mencari pola kesalahan yang sering muncul pada sistem, seperti pengurangan stok ganda, pencatatan mutasi yang hilang, atau kesalahan logika percabangan.

Contoh penerapan pada LocalesPro:

- Double Deduction: Memastikan stok tidak berkurang dua kali untuk satu transaksi yang sama (mencegah klik konfirmasi pembayaran berulang).
- Missing Movement: Memastikan setiap perubahan stok tercatat di tabel stock_movements.
- Branch Leakage: Memastikan stok antar cabang tidak tercampur ketika transaksi dilakukan di cabang berbeda.
- Broken Recipe: Memastikan resep yang tampil sesuai dengan resep yang dipakai untuk hitung stok.

2.6 GitHub sebagai Media Komunikasi dan Dokumentasi Pengujian

GitHub merupakan platform berbasis web yang digunakan untuk version control dan kolaborasi pengembangan perangkat lunak. Dalam konteks Software Quality Assurance (SQA), GitHub dimanfaatkan sebagai media komunikasi antara Programmer dan SQA Tester, serta sebagai tempat penyimpanan dan dokumentasi seluruh proses pengujian.

2.6.1 GitHub sebagai Media Komunikasi

GitHub menyediakan beberapa fitur yang memungkinkan komunikasi efektif antara Programmer dan SQA Tester, yaitu:

1. Issue: Digunakan untuk melaporkan bug, mengajukan pertanyaan, atau memberikan saran perbaikan. SQA Tester dapat membuat issue untuk setiap bug yang ditemukan, dan Programmer dapat merespons dengan komentar atau pull request perbaikan.
2. Pull Request (PR): Digunakan untuk mengusulkan perubahan kode. SQA Tester dapat mereview perubahan yang diajukan Programmer sebelum digabungkan ke branch utama.
3. Comment: Digunakan untuk berdiskusi pada baris kode tertentu, issue, atau pull request.
4. Mention (@username): Digunakan untuk memberi notifikasi kepada anggota tim terkait.

2.6.2 GitHub sebagai penyimpanan data pengujian

GitHub Repository dapat digunakan untuk menyimpan berbagai dokumen pengujian, antara lain Test Plan, Test Case, Test Report, serta bukti pendukung berupa screenshot dan log hasil pengujian. Repository ini juga berfungsi sebagai media kolaborasi, dokumentasi, dan pengelolaan versi dokumen selama proses Software Quality Assurance (SQA).

Pada penelitian ini, repository GitHub disusun ke dalam beberapa folder utama, yaitu Source Code, Dokumen SQA, dan Testing. Folder Testing digunakan untuk menyimpan seluruh dokumen pengujian dalam format Markdown (.md) yang dikelompokkan berdasarkan jenis pengujiannya, yaitu:

- 01-Whitebox-Testing/ — berisi dokumen pengujian *white box*, seperti *desk checking*, *code walkthrough*, *control flow testing*, *data flow testing*, dan *basic path testing*.
- 02-Blackbox-Testing/ — berisi dokumen pengujian *black box*, seperti *equivalence partitioning*, *boundary value analysis*, *decision table*, *cause-effect graphing*, dan *robustness testing*.
- 03-Greybox-Testing/ — berisi dokumen pengujian *grey box*, seperti *orthogonal array testing*, *matrix testing*, *regression testing*, dan *pattern testing*.

Selain itu, setiap folder pengujian dapat dilengkapi dengan bukti pendukung berupa *screenshot* dan dokumentasi hasil pengujian untuk memudahkan proses pelacakan, evaluasi, dan pengelolaan hasil pengujian.

2.6.3 Github Sebagai Dokumentasi Bug Fixing

Proses bug fixing dapat didokumentasikan melalui:

1. Issue Tracking: Setiap bug diberi label (bug, enhancement, critical) dan status (open, in progress, closed).
2. Commit Message: Programmer menuliskan perubahan yang dilakukan saat memperbaiki bug.

3. Version Control: Setiap perubahan dapat dilacak kembali ke versi sebelumnya jika diperlukan.

2.6.4 Keuntungan menggunakan Github untuk SQA

1. Transparansi: Seluruh tim dapat melihat status pengujian dan perbaikan secara real-time.
2. Akuntabilitas: Setiap tindakan tercatat dengan jelas (siapa yang membuat issue, siapa yang memperbaiki, kapan selesai).
3. Kolaborasi: Programmer dan SQA Tester dapat bekerja secara paralel tanpa mengganggu pekerjaan satu sama lain.
4. Dokumentasi Terstruktur: Seluruh proses pengujian tersimpan dalam satu repository yang dapat diakses kapan saja.

BAB III OBJEK DAN LINGKUNGAN PENGUJIAN

3.1 Deskripsi Aplikasi

LocalesPro merupakan aplikasi Point of Sale (POS) yang dikembangkan untuk membantu operasional usaha kafe atau penjualan minuman. Aplikasi ini memiliki beberapa fitur utama, seperti pengelolaan bahan baku, pengelolaan produk, pembuatan resep produk, transaksi penjualan, pengelolaan stok, mutasi stok, serta laporan transaksi. Seluruh fitur tersebut saling terhubung sehingga setiap aktivitas yang dilakukan pada satu modul akan memengaruhi modul lainnya.

Aplikasi ini memiliki dua role utama, yaitu admin dan kasir. Admin bertugas mengelola bahan baku, produk, resep, cabang, dan akun kasir. Kasir bertugas memproses transaksi penjualan melalui halaman POS.

3.2 Fitur yang diuji

Fokus pengujian adalah integrasi antara modul-modul berikut:

Modul	Peran dalam Pengujian
Bahan Baku	Menyimpan data bahan, satuan, stok awal, dan batas minimum
Menu & Resep	Menyimpan produk/menu beserta komposisi bahan per porsi
POS	Memproses transaksi penjualan produk oleh kasir
Stok	Menampilkan stok bahan setelah transaksi atau penambahan stok
Mutasi stok	Menampilkan riwayat stok masuk, stok keluar karena penjualan, dan stok masuk karena void
Laporan Transaksi	Menampilkan Transaksi paid dan void serta menjadi tempat void transaksi

3.3 Alur bisnis fitur bahan baku

Alur bisnis yang diuji adalah sebagai berikut:

1. Admin login ke aplikasi.

2. Admin membuat data bahan baku, misalnya Sedotan, Keju, dan Sirup Gula Aren.
3. Admin membuat produk/menu, misalnya Kopisusu.
4. Admin menambahkan resep bahan baku pada produk Kopisusu.
5. Kasir login dan membuka halaman POS.
6. Kasir memilih produk Kopisusu dan menentukan jumlah pembelian.
7. Sistem menghitung total pembayaran.
8. Kasir memasukkan uang tunai dan memproses transaksi.
9. Jika stok bahan cukup dan pembayaran valid, transaksi berhasil.
10. Sistem mengurangi stok bahan baku sesuai resep dikalikan qty pembelian.
11. Sistem mencatat riwayat mutasi stok keluar.
12. Jika transaksi di-void, stok bahan baku dikembalikan

3.4 Teknologi

Komponen	Teknologi
Frontend	React / Vite
Backend	PHP
Database	MySQL
Web Server	XAMPP
API Communication	Axios
Repository	GitHub

3.5 Source Code yang relevan

File	Fungsi
backend/api/products/create.php	Membuat produk dan resep
backend/api/products/update.php	Mengubah produk dan resep
backend/api/transactions/create.php	Memproses transaksi POS
backend/api/transactions/delete.php	Void transaksi

backend/config/payment_helpers.php	Validasi stok dan pengurangan stok
backend/config/inventory_helpers.php	Mencatat mutasi stok
frontend/scr/pages/pos/POS.jsx	Tampilan dan interaksi
frontend/scr/pages/stock/StockList.jsx	Tampilan stok dan mutasi

3.6 Struktur Repository GitHub

Repository GitHub digunakan sebagai media kolaborasi, komunikasi antara Programmer dan SQA Tester, serta penyimpanan dan dokumentasi seluruh proses pengujian. Struktur folder repository

Folder	Keterangan
01-Whitebox-Testing	Berisi dokumen pengujian white box, seperti desk checking, code walkthrough, control flow testing, data flow testing, dan basic path testing
02-Blackbox-Testing	Berisi dokumen pengujian black box, seperti equivalence partitioning, boundary value analysis , decision table, cause-effect graphing, dan robustness testing
03-Greybox-Testing	Berisi dokumen pengujian grey box, seperti orthogonal arrat testing, matrix testing, regression testing, dan pattern testing

3.7 Data uji global

Data uji yang digunakan secara konsisten pada seluruh dokumen pengujian:

Data	Nilai
Produk	Kopisusu
Harga	Rp 10.000
Qty transaksi normal	3 porsi
Total pembayaran normal	Rp 30.000
Uang diterima normal	Rp 50.000

Kembalian normal	Rp 20.000
------------------	-----------

Resep produk Kopisusu:

Bahan	Stok Awal	Kebutuhab per Porsi
Sedotan	10	1
Keju	1.000	100
Sirup Gula Aren	1.500	10

Expected stok setelah transaksi qty 3:

Bahan	Stok Awal	Total Terpakai	Stok Akhir
Sedotan	10 pcs	3 pcs	7 pcs
Keju	1.000 gr	300 gr	700 gr
Sirup Gula Aren	1.500 ml	30 ml	1.470 ml

3.8 Kriteria keberhasilan umum

1. Produk tidak dapat dibuat tanpa resep bahan baku.
2. Resep produk hanya menerima bahan dan jumlah kebutuhan yang valid.
3. Transaksi POS berhasil jika produk aktif, qty valid, stok cukup, cabang sesuai, dan pembayaran cukup.
4. Stok bahan baku berkurang sesuai resep saat transaksi berhasil.
5. Transaksi gagal tidak mengurangi stok bahan baku.
6. Riwayat mutasi stok tercatat saat stok berkurang karena penjualan.
7. Void transaksi paid mengembalikan stok bahan baku.
8. Sistem menolak input ekstrem atau tidak valid tanpa merusak data stok.

BAB IV PERANCANGAN PENGUJIAN

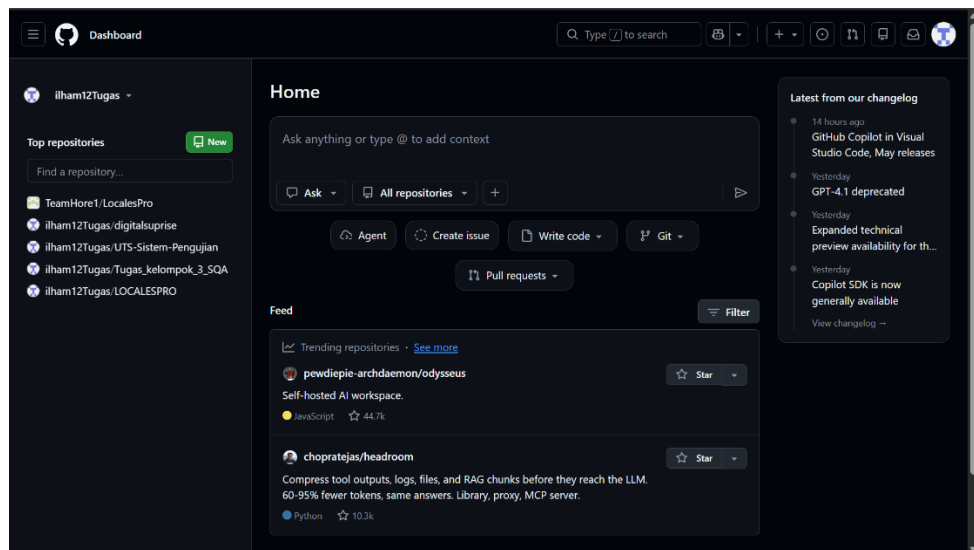
4.1 Pembuatan Repository Github

GitHub digunakan sebagai media kolaborasi dalam pengembangan aplikasi LocalesPro. Repository berfungsi sebagai tempat penyimpanan source code, dokumentasi, serta riwayat perubahan yang dilakukan selama proses pengembangan sistem.

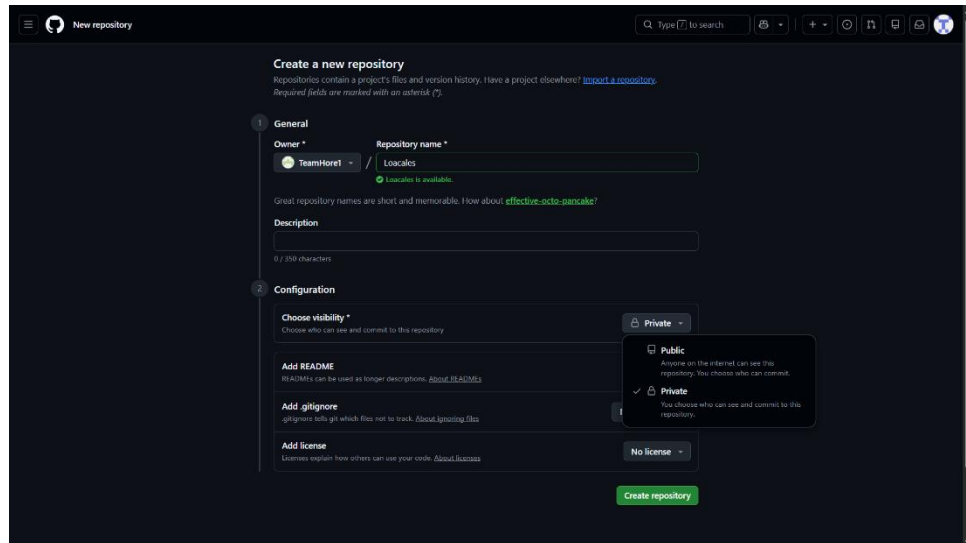
Tahapan pertama yang dilakukan adalah membuat repository baru dengan nama **LocalesPro**. Pada tahap ini ditentukan nama repository, visibilitas repository, serta pembuatan file README sebagai dokumentasi awal proyek.

Langkah-langkah yang dilakukan adalah sebagai berikut:

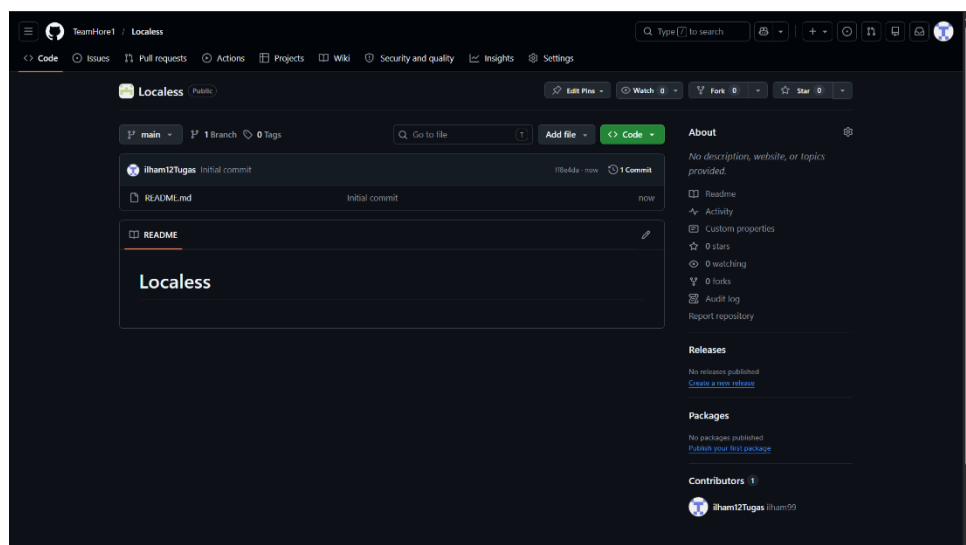
1. Login ke akun GitHub.
2. Memilih menu **New Repository**.



3. Mengisi nama repository **LocalesPro**.



4. Menentukan visibilitas repository.
5. Menambahkan file README.
6. Membuat repository.
7. Berhasil membuat repository



4.2 Penambahan Collaborator

Setelah repository berhasil dibuat, tahap berikutnya adalah menambahkan collaborator sebagai anggota tim yang terlibat dalam pengembangan aplikasi. Collaborator diberikan akses untuk berkontribusi dalam pengembangan sistem, melakukan perbaikan kode, dokumentasi, dan pengujian perangkat lunak.

Pengaturan hak akses bertujuan untuk menjaga keamanan repository serta menghindari perubahan yang tidak diinginkan.

Hak akses yang digunakan meliputi:

Role	Deskripsi
Admin	Memiliki akses penuh terhadap repository
Maintain	Mengelola repository tanpa akses administratif penuh
Write	Menambahkan file dan mengubah file
Read	Hanya dapat melihat repository

Tabel hak akses

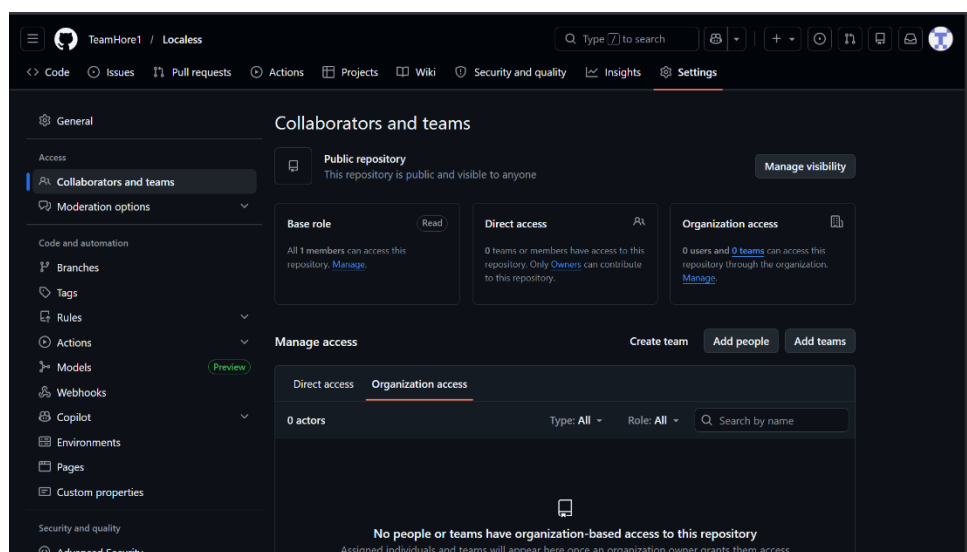
Contoh pembagian tugas anggota tim :

Anggota	Peran	Tugas
Develover	Admin	Mengembangkan
SQA Tester	Write	Melakukan pengujian sistem
Dokumentasi	Read	Mengelola dokumentasi

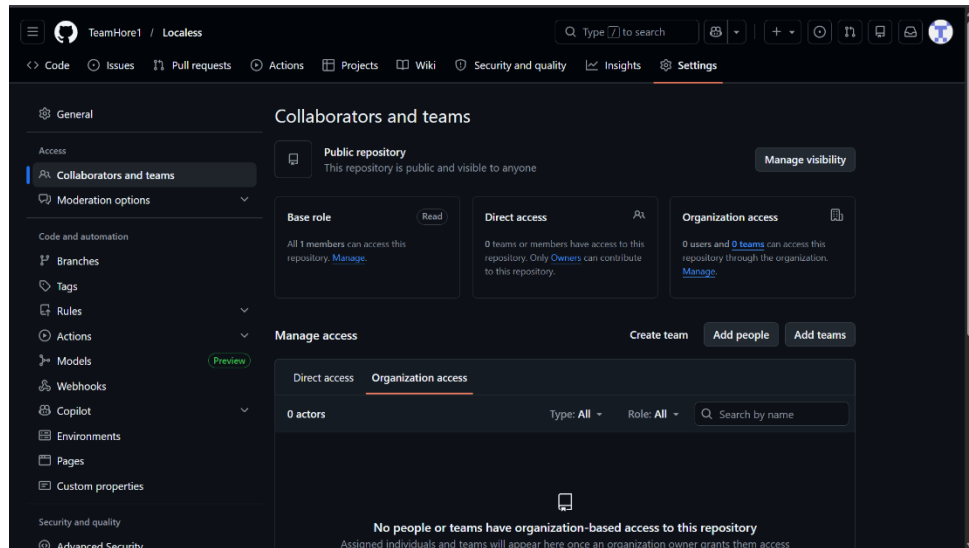
Tabel tugas anggota tim

Langkah-langkah yang dilakukan untuk menambahkan anggota dan mengelola hak akses adalah sebagai berikut:

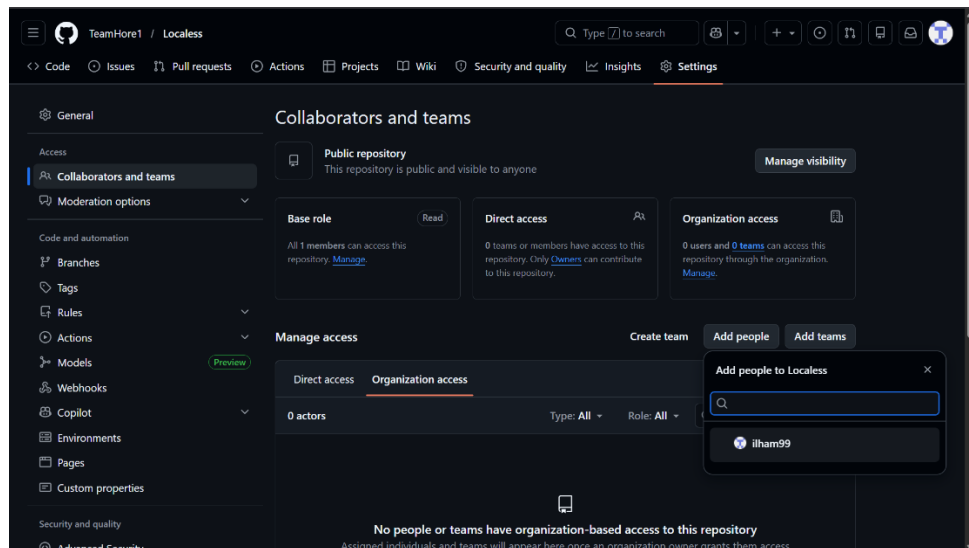
1. Membuka menu **Settings** pada repository.



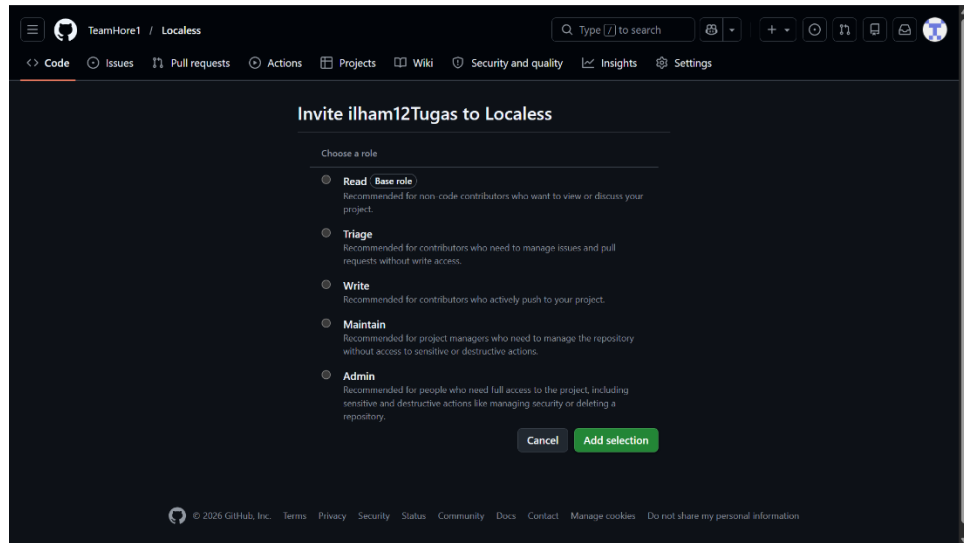
2. Memilih menu **Collaborators and teams**.



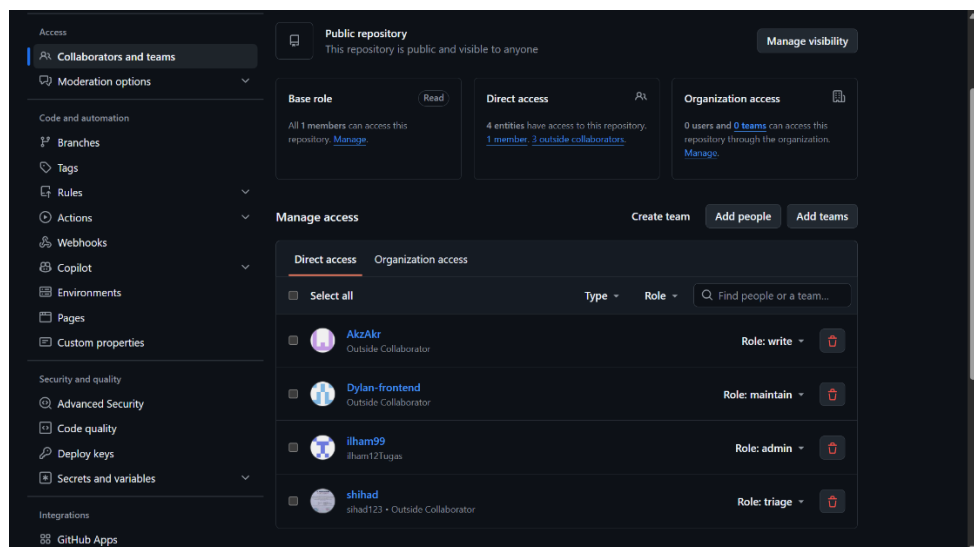
3. Menambahkan username anggota tim dengan click tombol Add People.



4. Atur Role dan akses lalu click Add Selection



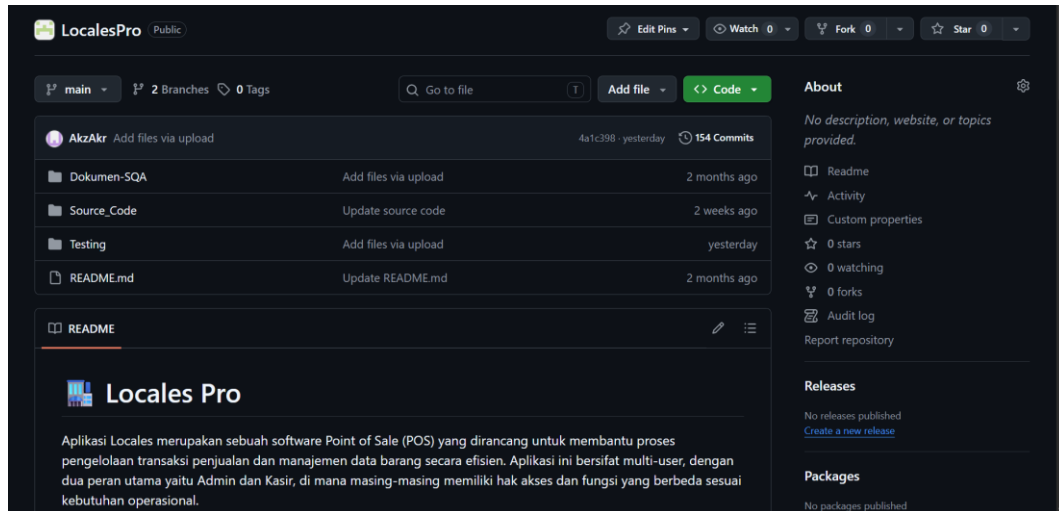
5. Mengirim undangan kolaborasi.
6. Menunggu anggota menerima undangan dan ketika sudah di terima maka akan menampilkan anggota yang sudah di invite beserta rolenya.



4.3 Pengelolaan File dan Commit Proyek

Repository digunakan untuk menyimpan seluruh file yang berkaitan dengan pengembangan aplikasi LocalesPro.

Struktur folder yang digunakan adalah sebagai berikut:



Dimana fungsi dari tiap folder adalah sebagai berikut :

Nama File / Folder	Keterangan
Dokumen SQA	Tempat menyimpan file file tentang proyek seperti SRS, dan lain lain.
Source_Code	Untuk menyimpan file kode mulai dari frontend dan backend dan konfigurasi lainnya.
Testing	Penyimpanan untuk menyimpan file dokkmentasi whitebox, blackbox, dan greybox testing.
README.md	Berisi Informasi Proyek yang sedang di kembangkan.

Tabel fungsi tiap folder dan file

Aktivitas yang dilakukan pada tahap ini meliputi:

1. Membuat folder proyek.
2. Menambahkan file baru.
3. Mengunggah file proyek.
4. Melakukan commit perubahan.
5. Menyimpan riwayat pengembangan sistem.

Commit digunakan untuk mendokumentasikan setiap perubahan yang dilakukan pada sistem.

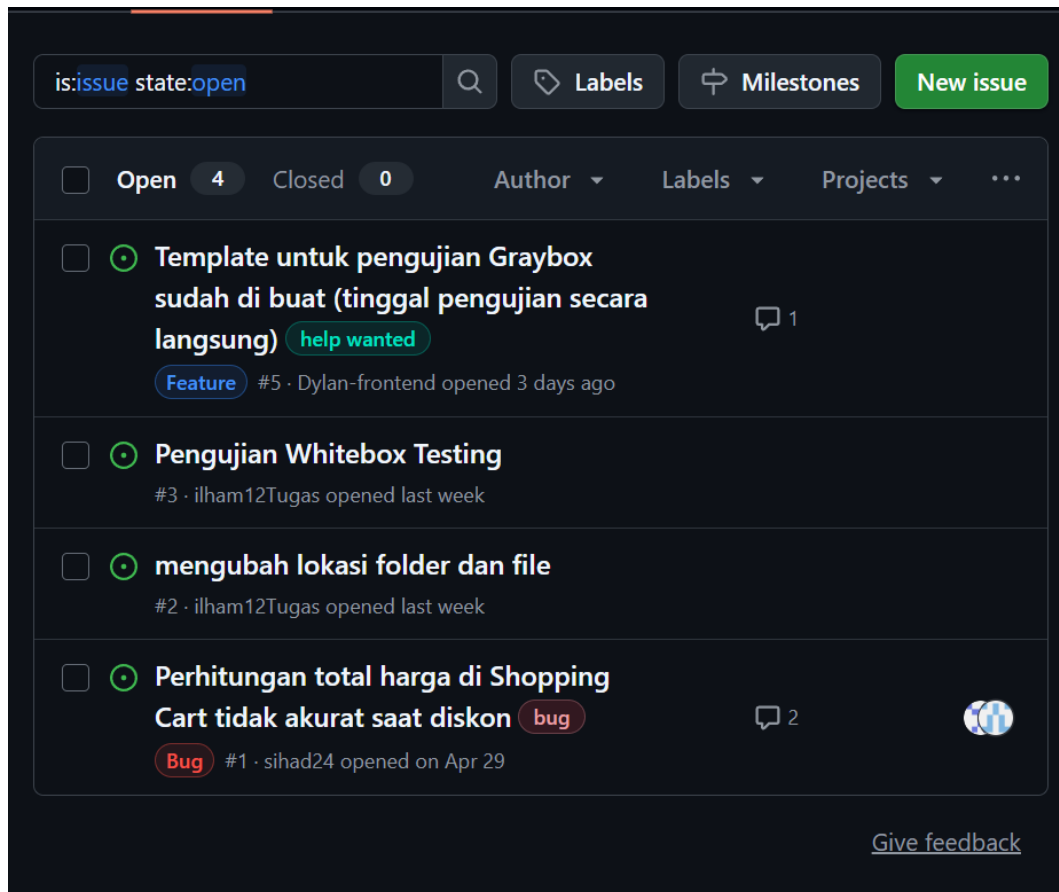
Contoh commit yang digunakan:

- Menambahkan fitur bahan baku.
- Menambahkan fitur resep produk.
- Menambahkan validasi stok.
- Memperbaiki bug transaksi POS.
- Memperbaiki mutasi stok.

4.4 Komunikasi antara Developer dan SQA Tester

Komunikasi antar anggota tim dilakukan untuk mempercepat proses identifikasi, pelaporan, dan perbaikan bug yang ditemukan selama pengembangan sistem. SQA Tester bertugas melakukan pengujian dan melaporkan permasalahan yang ditemukan, sedangkan Developer bertugas melakukan analisis dan perbaikan sistem.

Contoh alur komunikasi yang dilakukan adalah sebagai berikut.



4.5 Strategi Pengujian

Pengujian fitur bahan baku LocalesPro dilakukan dengan tiga pendekatan utama yang saling melengkapi. Masing-masing pendekatan memiliki fokus yang berbeda namun bertujuan sama, yaitu memastikan kualitas fitur pengurangan stok bahan baku otomatis berdasarkan resep produk.

Jenis Testing	Fokus	Metode yang digunakan
Black Box Testing	Input-output fitur bahan baku dari sudut pandang pengguna (admin dan kasir)	Equivalence Partitioning, Boundary value Analysis, Decision Table Testing, Cause-Effect Relationship Testing, Robustness Testing
White Box Testing	Source code, logika internal, percabangan,	Deck Checking, Code Walkthrough, Control

	alur data, dan jalur eksekusi backend	Flow Testing, Data Flow Testing, Basic Path Testing
Grey Box Testing	Kombinasi input-output dengan verifikasi struktur internal (API, database, helper backend)	Orthogonal Array Testing, Matrix Testing, Regression Tessting, Pattern Testing

Ketiga pendekatan ini dijalankan secara berurutan dan saling mendukung. Hasil black box testing membuktikan perilaku sistem dari sisi pengguna. Hasil white box testing membuktikan bahwa kode internal mendukung alur tersebut. Hasil grey box testing memperkuat kedua hasil tersebut dengan verifikasi data internal.

4.6 Lingkungan pnegujian

4.6.1 Perangkat keras

Komponen	Spesifikasi
Laptop / PC	Processor Intel Core i7 gen 3
RAM	6 GB
Penyimpanan	40 GB Free space
Koneksi internet	Diperlukan untuk aksews GitHub dan dokumentasi

4.6.2 Perangkat lunak

Komponen	Spesifikasi
Sistem Operasi	Windows 10 Pro
Web Server	XAMPP
Browser	Google Chrome / Mozilla Firefox
Database	MySQL
Code Editor	Visual Studio Code
Git	Untuk version control dan kolaborasi GitHub
API Client	Postman

4.6.3 Aplikasi yang diuji

Komponen	Keterangan
Nama Aplikasi	LocalesPro
Versi	Versi pengembangan semester ini
Frontend	React / Vire
Backend	PHP
Database	MySQL
Repository	GiHub

4.7 Data Uji

Data uji yang digunakan bersifat global dan konsisten untuk seluruh metode pnegujian agar hasil dapat dibandingkan dan ditelusuri.

4.7.1 Data Produk dan Transaksi

Data	Nilai
Produk uji	Kopisusu
Harga produk	Rp 10.000
Qty transaksi normal	3 porsi
Total tagihan normal	Rp 30.000
Uang diterima normal	Rp 50.000
Kembalian normal	Rp 20.000
Metode pembayaran	Cash / tunai

4.7.2 Data Resep produk

Bahan	Stok Awal	Kebutuhan per porsi
Sedotan	10 pcs	1
Keju	1.000 gr	100
Sirup Gula Aren	1.500 ml	10

4.7.3 Expected stok setelah transaksi

Bahan	Stok Awal	Total Terpakai	Stok akhir
Sedotan	10 pcs	3 pcs	7 pcs
Keju	1.000 gr	300 gr	700 gr

Sirup Gula Aren	1.500 ml	30 ml	1.470 ml
-----------------	----------	-------	----------

4.7.4 Kondisi Awal pengujian

Kondisi	Keterangan
Akun Admin	Dapat mengakses menu bahan baku, menu dan resep, stok dan laporan
Akun kasir	Dapat mengakses POS, Stok dan laporan Transaksi sesuai cabang
Cabang aktif	Cabang pengujian sudah aktif dan dipilih oleh user
Data bahan baku	Sedotan, Keju , Sirup Gula Aren
Data Produk valid	Active / aktif
Status transaksi berhasil	Paid
Status transaksi dibatalkan	Voided

4.8 Rencana pelaksanaan pengujian

4.8.1 Tahapan pengujian

Tahap	Kegiatan	Output
1	Persiapan lingkungan dan data uji	XAMPP berjalan, database siap, akun admin dan kasir tersedia
2	Black Box Testing	Dokumen pengujian per metode (EP, BVA, DT, CE, RT)
3	White Box Testing	Dokumen hasil review kode (DC, CW, CFT, DFT, BPT)
4	Grey Box Testing	Dokumen hasil verifikasi internal (OAT, MT, RG. PT)
5	Analisis dan perbandingan hasil	Tabel perbandingan temuan dari ketika pendekatan
6	Dokumentasi dan laporan akhir	Laporan akhir semester dalam format MS word

4.9 Kriteria Keberhasilan pengujian

4.9.1 Kriteria Keberhasilan Black Box

No	Kriteria
1	Produk tidak dapat dibuat tanpa resep bahan baku

2	Resep produk hanya menerima bahan dan jumlah kebutuhan yang valid
3	Transaksi POS berhasil jika produk aktif, qty valid, stok cukup, cabang sesuai, pembayaran cukup
4	Stok bahan baku berkurang sesuai resep saat transaksi
5	Transaksi gagal tidak mengurangi stok bahan baku
6	Riwayat mutasi stok tercatat saat stok berkurang karena penjualan
7	Void transaksi paid mengembalikan stok bahan baku
8	Sistem menolak input ekstrem atau tidak valid tanpa merusak data stok

4.9.2 Kriteria keberhasilan White box

No	Kriteria
1	Validasi resep produk sudah ada pada products/create.php
2	Validasi stok dilakukan sebelum transaksi commit pada payment_heloers.php
3	Pengurangan stok dilakukan setelah transaksi tersimpan
4	Mutasi stok dicatat melalui inventroty_helpers.php
5	Alur validasi dapat dipetakan dengan flow graph
6	Cyclomatic Complexity dapat dihitung dan jalur independen dapat diidentifikasi

4.9.3 Kriteria Keberhasilan Grey box

No	Kriteria
1	Stok dapat diverifikasi dari UI dan database
2	Mutasi sstok menjadi bukti internal perubahan data
3	Cabang perlu diperhatikan agar stok antar cabang tidak tercampur
4	Void perlu dicek agar tidak terjadi double restore
5	Kombinasi faktor dan level dapat diuji secara efisien
6	Perubahan fitur tidak merusak fitur yang sudah ada

4.10 Format Pencatatan hasil

4.10.1 Format hasil black box

ID test case	Expected Result	Actual Result	Status	Bukti /catatan
EP-01	Bahan berhasil disimpan	Diisi setelah pengujian	Pass / Fail / Blocked / Not run	Screenshot atau catatan hasil

Keterangan status:

- Pass: Hasil aktual sesuai expected result
- Fail: Hasil aktual tidak sesuai expected result
- Blocked: Test case belum dapat dijalankan karena data, akses, atau environment belum siap
- Not Run: Test case belum dijalankan

4.10.2 Format hasil white box

Komponen	Nilai Manual / Analisis	Nilai dari sistem / kode	Status	Catatan
Total transaksi	Rp 30.000	Diisi saat eksekusi	Pass / Fail	Cocok jika price * qty sama

4.10.3 Format Hasil Grey Box

ID Test Case	Expected Result	Actual Result	Status	Verifikasi Internal	Bukti
OAT	Transaksi berhasil dan stok berkurang	Diisi saat pengujian	Pass / Fail	Query tabel transaksi dan stok	Screenshot / Catatan

4.11 Traceability Matrix

Traceability Matrix menghubungkan kebutuhan sistem dengan metode pengujian yang digunakan:

Kebutuhan sistem	Black box	White box	Grey box
Produk wajib punya resep	EP-04, BVA-01, DT-03	DC (validasi resep)	OAT-02, MT-07
Stok berkurang otomatis	EP-06, CE-01, DT-01	CTF (jalur sukses), DTF (delta stok)	OAT-01, MT-01
Transaksi gagal tidak ubah stok	EP-07, CE-03, BVA-09	CTF (jalur gagal), BPT (P6)	MT-02, PT-BT-04
Void mengembalikan stok	CE-07, RB-17	DC (mode restore)	MT-10, PT-FD-05
Mutasi stok tercatat	EP-06, CE-01	DTF (stock_movements)	MT-01, RG-08
Validasi cabang	EP-12, CE-06	CFT (N9), BPT (P9)	OAT-07, MT-06

4.12 Dokumentasi dan pelaporan

Seluruh proses pengujian didokumentasikan dalam repository GitHub yang dibagi ke dalam tiga folder utama, yaitu Source Code, Dokumen SQA, dan Testing. Folder Testing digunakan untuk menyimpan seluruh dokumen dan hasil pengujian yang dikelompokkan berdasarkan metode pengujian yang digunakan

Folder	Isi
Source Code/	Berisi seluruh kode sumber (source code) aplikasi yang dikembangkan
Dokumen SQA/	Berisi dokumen software Quality Assurance, seperti perencanaan, analisis, dan dokumentasi kualitas perangkat lunak
Testing/	Berisi seluruh dokumen pengujian yang dikelompokkan berdasarkan metode pengujian

Struktur Folder dalam Testing:

Folder	Isi
01-Whitebox-Testing/	Dokumen analisis kode dan hasil pengujian white box
02-Blackbox-Testing/	Dokumen rancangan dan hasil pengujian black box
03-Greybox-Testing/	Dokumen verifikasi internal dan hasil pengujian grey box

Setiap dokumen pengujian menggunakan format Markdown (.md) agar dapat dibaca langsung di GitHub dan dilacak perubahannya melalui version control.

BAB V HASIL BLACK BOX TESTING

5.1 Ringkasan Black Box Testing

Black Box testing dilakukan untuk menguji fitur bahan baku pada aplikasi LocalesPro dari sudut pandang pengguna (admin dan kasir) tanpa melihat struktur internal kode program. Pengujian difokuskan pada proses bisnis utama, yaitu produk/menu memiliki resep bahan baku, kemudian saat produk dijual melalui POS, stok bahan baku otomatis berkurang sesuai resep dan jumlah pembelian.

Pengujian black box menggunakan lima metode: Equivalence Partitioning, Boundary Value Analysis, Decision Table Testing, Cause-Effect Relationship Testing, dan Robustness Testing. Seluruh test case dirancang berdasarkan data uji global yang konsisten, yaitu produk Kopisusu dengan harga Rp 10.000, qty

transaksi 3 porsi, dan resep bahan baku Sedotan 1 pcs, Keju 100 gr, Sirup Gula Aren 10 ml per porsi.

5.2 Metode yang Digunakan

5.2.1 Equivalence Partitioning

Equivalence Partitioning membagi data masukan ke dalam kelompok data valid dan tidak valid. Pengujian dilakukan dengan memilih perwakilan dari setiap kelompok data sehingga jumlah test case dapat dikurangi tanpa mengurangi efektivitas pengujian.

Objek Input	Partisi Valid	Bukan angka atau nilai negatif
Nama Bahan Baku	Terisi, contoh keju	Kosong
Stock bahan baku	Angka ≥ 0	Bukan angka atau nilai negatif
Satuan bahan baku	Terisi, gr, ml, pcs	Kosong
Nama produk	Terisi, contoh kopisusu	Kosong
Harga per produk	Angka lebih dari 0	Kosong, nol, negatif, atau bukan angka
Resep produk	Minimal 1 bahan	Resep kosong
Jumlah bahan resep	Lebih dari 0	Nol, negatif, kosong
Qty produksi di POS	Stock cukup	Nol, negatif, kosong
Stock banding kebutuhan	Stock cukup	Stock kurang
Uang tunai	Satu atau lebih dari total tagihan	Kurang dari total tagihan

Hasil pengujian :

ID	Kelas Partisi	Skenario	Input Uji	Expected Result	Actual Result	Status

EP - 01	Valid	Admin menambah bahan baku valid	Nama Keju, stok 1000, satuan gr, minimum 100	Bahan baku berhasil disimpan dan tampil di daftar bahan	Bahan baku berhasil disimpan dan tampil di daftar bahan	Pass
EP - 02	Tidak Valid	Admin menambah bahan tanpa nama	Nama kosong, stok 1000, satuan gr	Sistem menolak input dan bahan tidak tersimpan	Sistem menolak input dan bahan tidak tersimpan	Pass
EP - 03	Valid	Admin membuat produk dengan resep valid	Produk Kopisusu, harga 10000, resep Keju 100 gr	Produk berhasil disimpan dan resep tampil pada detail produk	Produk berhasil disimpan dan resep tampil pada detail produk	Pass
EP - 04	Tidak Valid	Admin membuat produk tanpa resep	Produk Kopisusu, harga 10000, resep kosong	Sistem menolak produk karena resep wajib diisi	Sistem menolak produk karena resep wajib diisi	Pass
EP - 05	Tidak Valid	Admin memasukkan jumlah bahan resep nol	Resep Keju 0 gr	Sistem menolak resep karena jumlah	Sistem menolak resep karena jumlah	Pass

				bahan tidak valid	bahan tidak valid	
EP - 06	Valid	Kasir menjual produk dengan stok cukup	Kopisusu qty 3, stok Keju 1000 gr	Transaksi berhasil, stok Keju berkurang menjadi 700 gr	Transaksi berhasil, stok Keju berkurang menjadi 700 gr	Pass
EP - 07	Tidak Valid	Kasir menjual produk saat stok kurang	Kopisusu qty 11, stok Sedotan 10 pcs	Transaksi ditolak, stok Sedotan tetap 10 pcs	Transaksi ditolak, stok Sedotan tetap 10 pcs	Pass
EP - 08	Tidak Valid	Kasir memproses POS tanpa item	Keranjang kosong	Sistem menolak transaksi karena item transaksi kosong	Sistem menolak transaksi karena item transaksi kosong	Pass
EP - 09	Valid	Kasir membayar tunai sesuai total	Total Rp 30.000, uang diterima Rp 30.000	Transaksi berhasil, kembalian Rp 0	Transaksi berhasil, kembalian Rp 0	Pass
EP - 10	Tidak Valid	Kasir membayar tunai kurang dari total	Total Rp 30.000, uang diterima Rp 20.000	Transaksi ditolak, stok bahan tidak berkurang	Transaksi ditolak, stok bahan tidak berkurang	Pass

EP - 11	Valid	Dua produk memakai bahan yang sama	Produk A dan Produk B sama-sama memakai Keju	Sistem mengakumulasi kebutuhan Keju dan mengurangi stok sesuai total pemakaian	Sistem mengakumulasi kebutuhan Keju dan mengurangi stok sesuai total pemakaian	Pass
EP - 12	Tidak Valid	Produk nonaktif dicoba dijual	Produk status inactive dipilih pada POS	Produk tidak boleh diproses sebagai transaksi berhasil dan stok bahan tidak berubah	Produk tidak boleh diproses sebagai transaksi berhasil dan stok bahan tidak berubah	Pass

5.2.2 Boundary Value Analysis

Boundary Value Analysis menguji nilai-nilai di sekitar batas input karena kesalahan cenderung terjadi pada nilai minimum, maksimum, atau tepat di sekitar batas validasi.

Nilai Batas yang di uji :

Objek	Batas Tidak Valid	Batas Valid	Di atas Batas
Jumlah bahan dalam resep	0 bahan	1 bahan	2 bahan
Jumlah bahan resep	0	0,1 atau 1 sesuai satuan	Lebih dari batas minimal
Qty produk POS	0	1	2

Stok vs kebutuhan	Kebutuhan - 1	Sama dengan kebutuhan	Kebutuhan + 1
Pembayaran tunai	Total - 1	Sama dengan total	Total + 1
Alasan void	2 karakter	3 karakter	Lebih dari 3 karakter

Hasil Pengujian Boundary Value Analysis :

ID	Objek Batas	Skenario	Input Uji	Expected Result	Actual Result	Status
BV A-01	Jumlah bahan resep	Produk dibuat tanpa bahan resep	Resep 0 bahan	Produk ditolak	Produk ditolak	Pass
BV A-02	Jumlah bahan resep	Produk dibuat dengan jumlah resep minimum	Resep berisi 1 bahan	Produk berhasil disimpan	Produk berhasil disimpan	Pass
BV A-03	Jumlah bahan resep	Produk dibuat dengan lebih dari satu bahan	Resep berisi 3 bahan	Produk berhasil disimpan	Produk berhasil disimpan	Pass
BV A-04	Jumlah kebutuhan bahan	Jumlah bahan	Keju 0 gr	Resep ditolak	Resep ditolak	Pass

		resep nol				
BV A- 05	Jumlah kebutuhan bahan	Jumlah bahan resep minimu m valid	Keju 1 gr	Resep diterima	Resep diterima	Pass
BV A- 06	Qty POS	Qty produk di bawah batas	Kopisus u qty 0	Transaksi ditolak	Transaksi ditolak	Pass
BV A- 07	Qty POS	Qty produk pada batas minimu m	Kopisus u qty 1	Transaksi dapat diproses jika stok dan pembayara n cukup	Transaksi dapat diproses jika stok dan pembayara n cukup	Pass
BV A- 08	Qty POS	Qty produk di atas batas minimu m	Kopisus u qty 2	Transaksi dapat diproses jika stok dan pembayara n cukup	Transaksi dapat diproses jika stok dan pembayara n cukup	Pass
BV A- 09	Stok bahan	Stok kurang 1 dari kebutuh an	Sedotan tersedia 2 pcs, kebutuh an 3 pcs	Transaksi ditolak, stok tetap 2 pcs	Transaksi ditolak, stok tetap 2 pcs	Pass

BV A- 10	Stok bahan	Stok sama dengan kebutuh an	Sedotan tersedia 3 pcs, kebutuh an 3 pcs	Transaksi berhasil, stok menjadi 0 pcs	Transaksi berhasil, stok menjadi 0 pcs	Pass
BV A- 11	Stok bahan	Stok lebih 1 dari kebutuh an	Sedotan tersedia 4 pcs, kebutuh an 3 pcs	Transaksi berhasil, stok menjadi 1 pcs	Transaksi berhasil, stok menjadi 1 pcs	Pass
BV A- 12	Pembayar an tunai	Uang kurang 1 rupiah	Total Rp 30.000, bayar Rp 29.999	Transaksi ditolak	Transaksi ditolak	Pass
BV A- 13	Pembayar an tunai	Uang sama dengan total	Total Rp 30.000, bayar Rp 30.000	Transaksi berhasil, kembalian Rp 0	Transaksi berhasil, kembalian Rp 0	Pass
BV A- 14	Pembayar an tunai	Uang lebih 1 rupiah	Total Rp 30.000, bayar Rp 30.001	Transaksi berhasil, kembalian Rp 1	Transaksi berhasil, kembalian Rp 1	Pass
BV A- 15	Void transaksi	Alasan void kurang dari batas tampilan	Alasan ok atau 2 karakter	Sistem frontend menolak konfirmasi void	Sistem frontend menolak konfirmasi void	Pass

BV A- 16	Void transaksi	Alasan void pada batas minimum	Alasan btl atau 3 karakter	Void dapat diproses, stok dikembalikan	Void dapat diproses, stok dikembalikan	Pass
----------------	----------------	--------------------------------	----------------------------	--	--	------

5.2.3 Decision Table Testing

Decision Table Testing menguji kombinasi kondisi yang mempengaruhi keputusan sistem saat kasir melakukan transaksi POS.

Kondisi dan Aksi :

Kode	Kondisi
C1	Produk berstatus aktif
C2	Produk memiliki resep bahan baku
C3	Qty produk valid, yaitu lebih dari 0
C4	Stok semua bahan resep mencukupi
C5	Produk dan bahan berada pada cabang aktif yang sesuai
C6	Pembayaran tunai sama atau lebih dari total tagihan

Kode	Aksi / Output
A1	Transaksi berhasil diproses
A2	Stok bahan baku berkurang sesuai resep
A3	Mutasi stok tipe penjualan tercatat
A4	Transaksi ditolak
A5	Stok bahan baku tidak berubah

Decision Table :

Rule	C1	C2	C3	C4	C5	C6	Expected Action
R1	Ya	Ya	Ya	Ya	Ya	Ya	A1, A2, A3
R2	Tidak	Ya	Ya	Ya	Ya	Ya	A4, A5
R3	Ya	Tidak	Ya	Ya	Ya	Ya	A4, A5
R4	Ya	Ya	Tidak	Ya	Ya	Ya	A4, A5
R5	Ya	Ya	Ya	Tidak	Ya	Ya	A4, A5
R6	Ya	Ya	Ya	Ya	Tidak	Ya	A4, A5
R7	Ya	Ya	Ya	Ya	Ya	Tidak	A4, A5
R8	Tidak	Tidak	Tidak	Tidak	Tidak	Tidak	A4, A5

Hasil pengujian Decision Table :

ID	Rule	Skenario	Input Uji	Expected Result	Actual Result	Status
DT-01	R1	Transaksi valid dengan stok cukup	Kopisusu aktif, resep lengkap, qty 3, uang Rp 50.000	Transaksi berhasil, stok Sedotan berkurang 3, Keju 300, Sirup 30	Transaksi berhasil, stok Sedotan berkurang 3, Keju 300, Sirup 30	Pass
DT-02	R2	Produk nonaktif dicoba dijual	Produk status inactive, qty 1, stok cukup,	Transaksi ditolak, stok bahan tidak berubah	Transaksi ditolak, stok bahan tidak berubah	Pass

			uang cukup			
DT-03	R3	Admin membuat produk tanpa resep	Produk valid tetapi resep kosong	Produk ditolak karena resep wajib, sehingga tidak dapat menjadi item POS valid	Produk ditolak karena resep wajib, sehingga tidak dapat menjadi item POS valid	Pass
DT-04	R4	Qty transaksi tidak valid	Produk aktif, qty 0, stok cukup, uang cukup	Transaksi ditolak karena item transaksi tidak valid	Transaksi ditolak karena item transaksi tidak valid	Pass
DT-05	R5	Salah satu bahan stok kurang	Sedotan tersedia 2 pcs, kebutuhan 3 pcs	Transaksi ditolak karena stok tidak cukup	Transaksi ditolak karena stok tidak cukup	Pass
DT-06	R6	Produk atau bahan berasal dari cabang berbeda	Produk cabang A diproses pada cabang B	Transaksi ditolak, stok tidak berubah	Transaksi ditolak, stok tidak berubah	Pass

DT-07	R7	Pembayaran tunai kurang	Total Rp 30.000, uang Rp 20.000	Transaksi ditolak, stok tetap	Transaksi ditolak, stok tetap	Pass
DT-08	R8	Semua kondisi tidak valid	Produk nonaktif, resep kosong, qty 0, stok kurang, uang kurang	Transaksi ditolak dan tidak ada perubahan stok	Transaksi ditolak dan tidak ada perubahan stok	Pass

5.2.4 Cause-Effect Relationship Testing

Cause-Effect Relationship Testing mengidentifikasi hubungan antara penyebab (cause) dan akibat (effect) yang dihasilkan oleh sistem.

Cause dan Effect :

Kode	Cause / Penyebab
C1	Produk memiliki resep bahan baku valid
C2	Qty produk di POS lebih dari 0
C3	Stok bahan baku mencukupi kebutuhan resep
C4	Pembayaran tunai cukup
C5	Produk berstatus aktif
C6	Produk dan bahan sesuai cabang aktif
C7	Transaksi paid diproses void
C8	Keranjang POS kosong

Kode	Effect / Akibat
E1	Transaksi berhasil dibuat
E2	Stok bahan baku berkurang sesuai resep
E3	Riwayat mutasi stok penjualan tercatat
E4	Transaksi ditolak
E5	Stok bahan baku tetap
E6	Stok bahan baku dikembalikan
E7	Status transaksi menjadi void

Hasil Pengujian Cause-Effect Relationship :

ID	Cause yang Diuji	Skenario	Input Uji	Expected Effect	Actual Effect	Status
C E- 01	C1+C2+C3+C4+C5+C6	Transaksi POS berhasil	Produk Kopisu aktif, resep valid, qty 3, stok cukup, uang Rp 50.000	Transaksi berhasil, stok bahan berkurang, mutasi stok tercatat	Transaksi berhasil, stok bahan berkurang, mutasi stok tercatat	Pass
C E- 02	C2 salah	Qty produk tidak valid	Produk aktif, qty 0, uang cukup	Transaksi ditolak, stok tidak berubah	Transaksi ditolak, stok tidak berubah	Pass

C E- 03	C3 salah	Stok bahan kurang	Sedotan tersedia 2 pcs, kebutuhan 3 pcs	Transaksi ditolak, stok Sedotan tetap 2 pcs	Transaksi ditolak, stok Sedotan tetap 2 pcs	Pass
C E- 04	C4 salah	Pembayaran tunai kurang	Total Rp 30.000, uang Rp 20.000	Transaksi ditolak, stok tidak berubah	Transaksi ditolak, stok tidak berubah	Pass
C E- 05	C5 salah	Produk nonaktif dijual	Produk status inactive	Transaksi ditolak, stok tidak berubah	Transaksi ditolak, stok tidak berubah	Pass
C E- 06	C6 salah	Produk cabang berbeda dijual	Produk cabang A diproses pada cabang B	Transaksi ditolak, stok tidak berubah	Transaksi ditolak, stok tidak berubah	Pass
C E- 07	C7	Void transaksi paid	Transaksi paid Kopisu qty 3 di-void	Status menjadi void, stok Sedotan +3, Keju +300,	Status menjadi void, stok Sedotan +3, Keju +300,	Pass

			dengan alasan valid	Sirup +30	Sirup +30	
C E-08	C8	Kasir memproses pembayaran tanpa item	Keranjang kosong	Transaksi ditolak, stok tidak berubah	Transaksi ditolak, stok tidak berubah	Pass
C E-09	C1 salah	Admin membuat produk tanpa resep	Produk baru tanpa bahan resep	Produk ditolak, tidak tersedia untuk transaksi valid	Produk ditolak, tidak tersedia untuk transaksi valid	Pass
C E-10	C1 benar dan dua produk berbagi bahan	Dua produk memakai Keju	Produk A qty 1, Produk B qty 2	Stok Keju berkurang berdasarkan total akumulasi kedua produk	Stok Keju berkurang berdasarkan total akumulasi kedua produk	Pass

5.2.5 Robustness Testing

Robustness Testing menguji kemampuan sistem menghadapi input tidak normal, ekstrem, tidak lengkap, atau aksi pengguna yang berpotensi menyebabkan data stok menjadi tidak konsisten.

Hasil Pengujian :

ID	Area	Skenario Tidak Normal	Input Uji	Expected Result	Actual Result	Stat us
RB -01	Bahan baku	Nama bahan kosong	Nama kosong, stok 1000, satuan gr	Sistem menolak input, bahan tidak tersimpan	Sistem menolak input, bahan tidak tersimpan	Pass
RB -02	Bahan baku	Stok awal negatif	Nama Keju, stok - 100, satuan gr	Sistem seharusnya menolak input agar stok bahan tidak bernilai negatif	Sistem seharusnya menolak input agar stok bahan tidak bernilai negatif	Pass
RB -03	Bahan baku	Satuan kosong	Nama Sirup, stok 1000, satuan kosong	Sistem seharusnya menolak input karena satuan wajib ada untuk perhitunga n stok	Sistem seharusnya menolak input karena satuan wajib ada untuk perhitunga n stok	Pass
RB -04	Produk	Harga produk nol	Produk Kopisus u, harga	Sistem seharusnya menolak	Sistem seharusnya menolak	Pass

			0, resep valid	harga nol agar transaksi tidak menghasilkan total tidak valid	harga nol agar transaksi tidak menghasilkan total tidak valid	
RB-05	Produk	Produk dibuat tanpa resep	Produk valid, resep kosong	Sistem menolak produk karena resep wajib diisi	Sistem menolak produk karena resep wajib diisi	Pass
RB-06	Resep	Jumlah bahan negatif	Keju - 100 gr	Sistem menolak resep tidak valid	Sistem menolak resep tidak valid	Pass
RB-07	Resep	Bahan resep tidak dipilih	Amount 100, ingredient kosong	Sistem menolak penyimpanan resep dan meminta bahan dipilih	Sistem menolak penyimpanan resep dan meminta bahan dipilih	Pass
RB-08	POS	Keranjang kosong dibayar	Tidak ada item	Sistem menolak transaksi kosong	Sistem menolak transaksi kosong	Pass
RB-09	POS	Qty produk nol	Kopisusu qty 0	Sistem menolak transaksi	Sistem menolak transaksi	Pass

RB -10	POS	Qty sangat besar melebihi stok	Kopisus u qty 9999	Sistem menolak karena stok bahan tidak cukup	Sistem menolak karena stok bahan tidak cukup	Pass
RB -11	POS	Salah satu bahan habis	Sedotan 0 pcs, bahan lain cukup	Produk ditandai habis atau transaksi ditolak	Produk ditandai habis atau transaksi ditolak	Pass
RB -12	Pembayar an	Uang tunai nol	Total Rp 30.000, bayar Rp 0	Sistem menolak pembayara n atau tidak memproses transaksi valid	Sistem menolak pembayara n atau tidak memproses transaksi valid	Pass
RB -13	Pembayar an	Uang tunai kurang	Total Rp 30.000, bayar Rp 20.000	Transaksi ditolak, stok tidak berubah	Transaksi ditolak, stok tidak berubah	Pass
RB -14	Pembayar an	Uang tunai sangat besar	Total Rp 30.000, bayar Rp 1.000.00 0	Transaksi berhasil, kembalian dihitung benar	Transaksi berhasil, kembalian dihitung benar	Pass
RB -15	Cabang	Produk dari	Produk cabang	Transaksi ditolak,	Transaksi ditolak,	Pass

		cabang berbeda dijual	A, user cabang B	stok tidak berubah	stok tidak berubah	
RB-16	Void	Void transaksi dengan ID tidak valid	ID transaksi 0	Sistem menolak ID transaksi tidak valid	Sistem menolak ID transaksi tidak valid	Pass
RB-17	Void	Void transaksi yang sudah void	Transaksi status Voided di-void lagi	Sistem menolak proses void ulang dan menampilkan pesan transaksi sudah void	Sistem menolak proses void ulang dan menampilkan pesan transaksi sudah void	Pass
RB-18	Aksi berulang	Tombol konfirmasi pembayaran ditekan berulang	Klik konfirmasi beberapa kali saat proses berjalan	Sistem hanya membuat satu transaksi dan stok hanya berkurang satu kali	Sistem hanya membuat satu transaksi dan stok hanya berkurang satu kali	Pass

5.3 Ringkasan Hasil Black Box

Metode	Jumlah Test Case	Fokus	Status
--------	------------------	-------	--------

Equivalence Partitioning	12	Input valid/tidak valid	Pass
Boundary Value Analysis	16	Nilai batas	Pass
Decision Table Testing	8	Kombinasi kondisi	Pass
Cause-Effect Relationship	10	Sebab-akibat	Pass
Robustness Testing	18	Input tidak normal	Pass
Total	64		

5.4 Temuan Black Box

Berdasarkan hasil pengujian black box yang telah dilakukan, seluruh test case menunjukkan status Pass, yang berarti sistem telah berhasil menangani berbagai skenario pengujian. Berikut temuan utama dari pengujian:

1. Validasi resep produk wajib — Sistem berhasil menolak produk tanpa resep, sehingga tidak ada produk jual tanpa pengurangan stok.
2. Validasi stok bahan baku negatif — Sistem berhasil menolak input stok negatif sejak awal, sehingga data inventory tetap akurat.
3. Validasi harga produk nol atau negatif — Sistem berhasil menolak harga produk nol, sehingga transaksi selalu menghasilkan total yang valid.
4. Validasi qty transaksi — Sistem berhasil menolak qty nol atau negatif sebelum perhitungan stok dilakukan.
5. Validasi pembayaran kurang — Sistem berhasil menolak transaksi dengan uang kurang dan stok tidak berubah.
6. Validasi double void — Sistem berhasil menolak void ulang pada transaksi yang sudah void, sehingga stok tidak kembali dua kali.

7. Validasi klik pembayaran berulang — Sistem berhasil mencegah transaksi ganda dengan hanya membuat satu transaksi meskipun tombol ditekan berulang.
8. Validasi produk lintas cabang — Sistem berhasil menolak produk dan bahan dari cabang berbeda, sehingga data stok antar cabang tidak tercampur.

BAB VI HASIL WHITE BOX TESTING

6.1 Ringkasan White Box testing

White Box Testing dilakukan untuk menguji struktur internal perangkat lunak, source code, percabangan program, aliran data, serta jalur eksekusi yang digunakan oleh sistem pada fitur bahan baku LocalesPro. Pengujian ini berfokus pada backend API dan helper stok yang menjadi inti proses transaksi POS dan pengurangan stok bahan baku otomatis.

Berbeda dengan Black Box Testing yang hanya mengamati input dan output, White Box Testing mengharuskan penguji memahami logika dan struktur kode yang digunakan dalam pengembangan sistem. Pengujian dilakukan pada file `backend/api/transactions/create.php`, `backend/config/payment_helpers.php`, dan `backend/config/inventory_helpers.php`.

6.2 Metode yang Digunakan

6.2.1 Desk Checking

Desk Checking merupakan metode pemeriksaan kode program secara manual untuk memastikan bahwa logika program dan nilai variabel berjalan sesuai dengan yang diharapkan.

File yang di Periksa :

File	Fungsi / Bagian Kode	Peran
<code>backend/api/transactions/create.php</code>	Validasi item, produk, cabang, pembayaran, simpan transaksi	Entry point transaksi POS
<code>backend/config/payment_helpers.php</code>	<code>validateInventoryAvailabilityForCart()</code>	Validasi ketersediaan stok bahan
<code>backend/config/payment_helpers.php</code>	<code>applyInventoryUsageForTransaction()</code>	Mengurangi atau mengembalikan stok bahan
<code>backend/config/inventory_helpers.php</code>	<code>recordStockMovement()</code>	Mencatat riwayat mutasi stok

Potongan Kode Pertama :

```
// Validasi item transaksi
foreach ($data->items as $item) {
    $productId = (int) ($item->id ?? 0);
    $quantity = (int) ($item->qty ?? 0);

    if ($productId <= 0 || $quantity <= 0) {
        throw new InvalidArgumentException("Ada item
transaksi yang tidak valid.");
    }
}

// Perhitungan kebutuhan bahan
$usageByIngredient[$ingredientId]["required"] +=
$requiredPerItem * $quantity;

// Validasi stok cukup
if ($required > $available) {
    throw new InvalidArgumentException(
        "Stok {$ingredientName} tidak cukup. Tersedia
{$available} {$sunit}, butuh {$required} {$sunit}."
    );
}

// Pengurangan stok bahan
$direction = $mode === "restore" ? 1 : -1;
$delta = round($direction * $totalUsage, 2);
$stockAfter = round($stockBefore + $delta, 2);
```

Data Uji Desk Checking :

Data	Nilai
Produk	Kopisusu
Qty transaksi	3
Harga produk	Rp 10.000
Total transaksi	Rp 30.000
Pembayaran tunai	Rp 50.000
Kembalian	Rp 20.000

Resep Produk :

Bahan	Stok Awal	Kebutuhan Per Produk
Sedotan	10 pcs	1 pcs

Keju	1.000 gr	100 gr
Sirup Gula Aren	1.500 ml	10 ml

Tabel Desk Checking :

Langka h	Kode / proses	Nilai Varabel	Expected Resault	Actual Resault	Statu s
1	Input items diterima	items = [{id: Kopisusu, qty: 3}]	Item tidak kosong	Item tidak kosong	Pass
2	Validasi item	productId > 0, quantity = 3	Item valid	Item valid	Pass
3	Ambil produk	status = active, branch_id = cabang aktif	Produk boleh dijual	Produk boleh dijual	Pass
4	Hitung subtotal	10000 * 3 = 30000	totalPrice = 30000	totalPrice = 30000	Pass
5	Validasi pembayara n	amountPaid = 50000, totalPrice = 30000	Pembayara n cukup	Pembayara n cukup	Pass
6	Validasi resep Sedotan	1 * 3 = 3, tersedia 10	Stok cukup	Stok cukup	Pass
7	Validasi resep Keju	100 * 3 = 300, tersedia 1000	Stok cukup	Stok cukup	Pass
8	Validasi resep Sirup	10 * 3 = 30, tersedia 1500	Stok cukup	Stok cukup	Pass
9	Simpan transaksi	payment_statu s = Paid	Transaksi tercatat	Transaksi tercatat	Pass

10	Kurangi stok Sedotan	$10 - 3 = 7$	Stok akhir 7 pcs	Stok akhir 7 pcs	Pass
11	Kurangi stok Keju	$1000 - 300 = 700$	Stok akhir 700 gr	Stok akhir 700 gr	Pass
12	Kurangi stok Sirup	$1500 - 30 = 1470$	Stok akhir 1470 ml	Stok akhir 1470 ml	Pass
13	Catat mutasi stok	movement_ty pe = sale, direction = out	Riwayat stok keluar tercatat	Riwayat stok keluar tercatat	Pass

Skenario Gagal Yang di cek Manual :

Skenario	Nilai Variabel	Expected Resault	Actual Resault	Status
Item kosong	items = []	Response error, transaksi tidak dibuat	Response error, transaksi tidak dibuat	Pass
Qty tidak valid	qty = 0	Throw Ada item transaksi yang tidak valid.	Throw Ada item transaksi yang tidak valid.	Pass
Stok kurang	Sedotan tersedia 2, kebutuhan 3	Throw error stok tidak cukup	Throw error stok tidak cukup	Pass
Pembayaran kurang	amountPaid = 20000, totalPrice = 30000	Throw error uang tunai kurang	Throw error uang tunai kurang	Pass

Produk beda cabang	product.branch_id != branchId	Throw error produk tidak tersedia untuk cabang aktif	Throw error produk tidak tersedia untuk cabang aktif	Pass
--------------------	-------------------------------	--	--	------

6.2.2 Code Walkthrough

Code Walkthrough merupakan proses peninjauan kode program secara sistematis untuk mengidentifikasi kesalahan logika, struktur, dan implementasi kode.

File dan Fungsi yang Diriview :

File	Fungsi / Bagian	Hasil Walkthrough
backend/api/products/create.php	Membuat produk dan resep	Produk wajib memiliki minimal 1 bahan resep
backend/api/products/update.php	Update produk dan resep	Resep lama dapat diupdate dan bahan yang dihapus ikut dihapus
backend/api/transactions/create.php	Membuat transaksi POS	Transaksi memvalidasi item, produk, cabang, pembayaran, dan stok
backend/config/payment_helpers.php	Validasi stok dan pengurangan stok	Kebutuhan bahan dihitung dari resep dan qty transaksi
backend/api/transactions/delete.php	Void transaksi	Transaksi paid dapat diubah menjadi void dan stok dikembalikan

backend/config/inventories_helpers.php	Mutasi stok	Setiap perubahan stok dicatat dalam stock_movements
--	-------------	---

Traceability Walkthrough :

Kebutuhan Sistem	Potongan Kode yang Direview	Kesimpulan walkthrough	Status
Produk wajib punya resep	Validasi \$recipe pada products/create.php	Produk tanpa resep dihentikan sebelum insert	Pass
Item POS tidak boleh kosong	Validasi empty(\$data->items)	Transaksi kosong ditolak	Pass
Produk harus aktif	Validasi status produk	Produk nonaktif tidak dapat menjadi transaksi valid	Pass
Pembayaran cash harus cukup	Validasi \$amountPaid < \$totalPrice	Pembayaran kurang menghentikan transaksi	Pass
Stok harus cukup	validateInventoryAvailabilityForCart()	Stok dicek sebelum transaksi commit	Pass
Stok berkurang	applyInventoryUsageForTransaction(..., "deduct")	Pengurangan stok dilakukan setelah transaksi tersimpan	Pass

Potensi Temuan dari Walkthrough :

No	Temuan	Risiko	Rekomendasi	Status
1	Harga produk pada create hanya dicek is_numeric	Harga 0 atau negatif berpotensi lolos	Tambahkan validasi price > 0	Perlu perbaikan
2	Produk tanpa resep lama dapat menjadi risiko jika ada data warisan	Stok tidak berkurang jika produk lama tidak punya resep	Audit data produk lama	Perlu perhatian
3	Pengurangan stok bergantung pada data transaction_items	Jika item gagal tersimpan, stok tidak boleh diproses	Sudah dilindungi transaksi database	Pass
4	Pencatatan mutasi stok silent return jika data tidak valid	Mutasi bisa tidak tercatat jika ingredient/branch/quantity invalid	Pastikan data mutasi lengkap sebelum pemanggilan	Perlu perhatian

6.2.3 Control Flow Testing

Control Flow Testing menganalisis alur eksekusi program berdasarkan node dan edge yang terdapat pada flow graph.

Skenario Control Flow :

ID	Jalur	Kondisi	Expected Result	Actual Result	Status
CF-01	N1-N2- N3-N4- N5-N6- N7-N8- N9-N10- N11-N12- N13-N14- N15-N16- N17-N18	Semua input valid, stok cukup	Transaksi berhasil dan stok berkurang	Transaksi berhasil dan stok berkurang	Pass
CF-02	N1-N2- N19	Item kosong	Response error, transaksi tidak dibuat	Response error, transaksi tidak dibuat	Pass
CF-03	N1-N2- N3-N4- N5-N6- N19	Qty atau product ID tidak valid	Rollback, stok tidak berubah	Rollback, stok tidak berubah	Pass
CF-04	N1-N2- N3-N4- N5-N6- N7-N8- N19	Produk tidak aktif	Rollback, stok tidak berubah	Rollback, stok tidak berubah	Pass
CF-05	N1-N2- N3-N4- N5-N6- N7-N8- N9-N19	Produk beda cabang	Rollback, stok tidak berubah	Rollback, stok tidak berubah	Pass

CF-06	N1-N2- N3-N4- N5-N6- N7-N8- N9-N10- N11-N12- N19	Uang tunai kurang	Rollback, stok tidak berubah	Rollback, stok tidak berubah	Pass
-------	--	-------------------------	------------------------------------	------------------------------------	------

6.2.4 Data Flow Testing :

Data Flow Testing menganalisis bagaimana data didefinisikan, di gunakan, dan dimodifikasi selama proses eksekusi program berlangsung.

Data Flow Test Case :

ID	Definisi Data	Penggunaan Data	Expected Result	Actual Result	Status
DF-01	\$quantity = 3	Menghitung subtotal dan kebutuhan bahan	Total harga dan kebutuhan bahan sesuai qty 3	Total harga dan kebutuhan bahan sesuai qty 3	Pass
DF-02	\$requiredPerItem = 100 untuk Keju	$100 * 3$	\$required = 300	\$required = 300	Pass
DF-03	\$available = 1000, \$required = 300	Validasi stok	Stok cukup, transaksi lanjut	Stok cukup, transaksi lanjut	Pass
DF-04	\$available = 200, \$required = 300	Validasi stok	Error stok kurang	Error stok kurang	Pass

DF -05	\$direction = -1, \$totalUsage = 300	Hitung delta	\$delta = - 300, stok berkurang	\$delta = - 300, stok berkurang	Pass
DF -06	\$direction = 1, \$totalUsage = 300	Void transaksi	\$delta = 300, stok dikembalika n	\$delta = 300, stok dikembalika n	Pass

6.2.5 Basic Path Testing

Basic Path Testing mengidentifikasi seluruh jalur independen dalam program berdasarkan nilai Cyclomatic Complexity.

Test Case Basic Path :

ID	Path	Input / Kondisi	Expected Result	Actual Result	Status
BP-01	P1	branchId = 0	Error Cabang aktif belum valid.	Error Cabang aktif belum valid.	Pass
BP-02	P2	productId = 0 atau qty = 0	Error item transaksi tidak valid	Error item transaksi tidak valid	Pass
BP-03	P3	Produk tidak memiliki resep	Tidak ada kebutuhan bahan yang dihitung	Tidak ada kebutuhan bahan yang dihitung	Pass
BP-04	P4	ingredientId <= 0 atau quantity_needed <= 0	Error resep produk tidak valid	Error resep produk tidak valid	Pass
BP-05	P5	Ingredient dari cabang berbeda	Error resep memakai	Error resep memakai	Pass

			bahan cabang berbeda	bahan cabang berbeda	
BP-06	P6	Stok Keju 200 gr, kebutuhan 300 gr	Error stok tidak cukup	Error stok tidak cukup	Pass
BP-07	P7	Satu bahan, stok cukup	Validasi selesai tanpa error	Validasi selesai tanpa error	Pass
BP-08	P8	Tiga bahan resep, stok semua cukup	Validasi selesai tanpa error	Validasi selesai tanpa error	Pass
BP-09	P9	Dua produk memakai Keju	Kebutuhan Keju diakumulasi	Kebutuhan Keju diakumulasi	Pass
BP-10	P10	Beberapa item transaksi valid	Semua item divalidasi	Semua item divalidasi	Pass
BP-11	P11	Transaksi berhasil mode deduct	Stok berkurang dan mutasi stok tercatat	Stok berkurang dan mutasi stok tercatat	Pass

6.3 Cyclomatic Complexity

Berdasarkan perhitungan pada fungsi `validateInventoryAvailabilityForCart()` menggunakan rumus flow graph:

$$V(G) = E - N + 2P$$

Simbol	Nilai	Keterangan
E	32	Jumlah edge pada flow graph

N	23	Jumlah node pada flow graph
P	1	Jumlah connected component

$V(G) = 32 - 23 + 2(1) = 11$. Hasil perhitungan menunjukkan bahwa terdapat 11 jalur independen yang harus diuji untuk mencapai 100% coverage pada fungsi validasi stok.

6.4 Ringkasan Hasil White Box

Metode	Fokus	Hasil	Status
Desk Checking	Nilai variabel stok	Sesuai perhitungan manual	Pass
Code Walkthrough	Review kode validasi	Alur jelas, validasi berurutan	Pass
Control Flow	Jalur sukses/gagal	7 skenario control flow berhasil diuji	Pass
Data Flow	Aliran data stok	Data dapat ditelusuri sampai mutasi stok	Pass
Basic Path	Jalur independen	$V(G) = 11$, 11 jalur berhasil diuji	Pass

6.5 Temuan White Box

1. Validasi resep sudah ada — Produk wajib memiliki minimal 1 bahan resep, dihentikan sebelum insert ke database.
2. Validasi stok dilakukan sebelum transaksi commit — Fungsi `validateInventoryAvailabilityForCart()` menjadi gerbang utama sebelum transaksi disimpan.
3. Pengurangan stok dilakukan setelah transaksi tersimpan — `applyInventoryUsageForTransaction()` dipanggil setelah transaksi dan item

transaksi berhasil disimpan.

4. Mutasi stok dicatat melalui helper — `recordStockMovement()` mencatat setiap perubahan stok dengan `movement_type` dan `direction`.
5. Rollback terjadi ketika `exception` — Transaksi database menggunakan `try-catch` sehingga data tetap konsisten saat terjadi error.
6. Potensi perbaikan — Validasi harga produk hanya menggunakan `is_numeric`, perlu ditambahkan `price > 0` untuk mencegah harga nol atau negatif.

6.6 Kesimpulan

White Box Testing pada fitur bahan baku LocalesPro berhasil membuktikan bahwa kode internal mendukung alur transaksi yang aman. Validasi dilakukan secara berurutan mulai dari item transaksi, produk aktif, cabang sesuai, pembayaran cukup, stok mencukupi, baru kemudian transaksi disimpan dan stok dikurangi.

Cyclomatic Complexity sebesar 11 menunjukkan bahwa fungsi `validateInventoryAvailabilityForCart()` memiliki kompleksitas sedang dengan 11 jalur independen yang perlu diuji. Seluruh jalur telah diidentifikasi dan diuji dengan status Pass.

Hasil white box testing ini memperkuat hasil black box testing sebelumnya, bahwa sistem mampu membedakan transaksi valid dan tidak valid, serta menjaga integritas stok bahan baku.

BAB VII

7.1 Ringkasan Grey Box Testing

Grey Box Testing dilakukan untuk menggabungkan pendekatan Black Box Testing dan White Box Testing pada fitur bahan baku LocalesPro. Pengujian dilakukan dengan menguji input dan output aplikasi dari sisi pengguna (admin dan kasir), sambil menggunakan pengetahuan terbatas tentang struktur internal sistem seperti endpoint API, tabel database, dan helper backend.

Tujuan grey box testing adalah memverifikasi bahwa hasil yang terlihat pada antarmuka pengguna (UI) sesuai dengan data internal yang tersimpan di database. Pengujian ini menjadi penghubung antara hasil black box yang membuktikan perilaku sistem dari sisi pengguna, dan hasil white box yang membuktikan logika internal kode.

7.2 Metode yang digunakan

7.2.1 Orthogonal Array Testing

Orthogonal Array Testing merupakan metode pengujian yang digunakan untuk menguji berbagai kombinasi faktor dan level secara efisien tanpa harus menguji seluruh kemungkinan kombinasi yang ada.

Faktor	Level 1	Level 2	Level 3
F1 – Status Produk	Aktif	Nonaktif	Produk beda cabang
F2 – Resep produk	Valid	Kosong	Jumlah bahan tidak valid
F3 – Kondisi Stok	Cukup	Sama dengan kebutuhan	Kurang
F4 – Qty Transaksi	1	3	0 / tidak valid
F5 – Pembayaran	Pas total	Lebih dari total	Kurang dari total

Dengan 5 faktor dan 3 level, exhaustive testing akan menghasilkan $3^5 = 243$ kombinasi. Dengan Orthogonal Array L9, kombinasi dapat direpresentasikan dengan 9 test case utama

Orthogonal Array L9:

TC	F1 - Status Produk	F2 – Resep Produk	F3 – Kondisi Stok	F4 - Qty	F5 - Pembayaran
OAT-01	Aktif	Valid	Cukup	1	Pas total
OAT-02	Aktif	Kosong	Sama kebutuhan	3	Lebih dari total
OAT-03	Aktif	Jumlah bahan tidak valid	Kurang	0	Kurang
OAT-04	Nonaktif	Valid	Sama kebutuhan	0	Kurang
OAT-05	Nonaktif	Kosong	Kurang	1	Pas total
OAT-06	Nonaktif	Jumlah bahan tidak valid	Cukup	3	Lebih dari total
OAT-07	Produk beda cabang	Valid	Kurang	3	Pas total
OAT-08	Produk beda cabang	Kosong	Cukup	0	Lebih dari total
OAT-09	Produk beda cabang	Jumlah bahan tidak valid	Sama kebutuhan	1	Kurang

Hasil pengujian :

ID	Skenario	Expected Result	Actual Result	Stat us	Verifikasi internal

OA T- 01	Produk aktif, resep valid, stok cukup, qty 1, bayar pas	Transaksi berhasil, stok berkurang 1 porsi	Transaksi berhasil, stok berkurang 1 porsi	Pass	transactions.payment_status = Paid, stok berkurang, mutasi sale tercatat
OA T- 02	Produk aktif tetapi resep kosong	Produk tidak boleh menjadi transaksi vallid	Produk tidak boleh menjadi transaksi vallid	Pass	Produk ditolak saat create/update, stok tidak berubah
OA T- 03	Resep tidak valid, qty 0, pembayaran kurang	Transaksi ditolak	Transaksi ditolak	Pass	Tidak ada transaksi paid dan stok tetap
OA T- 04	Produk nonaktif, stok sama kebutuhannya, qty 0	Transaksi ditolak	Transaksi ditolak	Pass	Response produk tidak tersedia / item invalid
OA T- 05	Produk nonaktif dan stok kurang	Transaksi ditolak	Transaksi ditolak	Pass	Stok tidak berubah
OA T- 06	Produk nonaktif walaupun	Transaksi ditolak	Transaksi ditolak	Pass	Tidak ada mutasi sale

	stok cukup dan bayar lebih				
OA T-07	Produk beda cabang dengan stok kurang	Transaksi ditolak karena cabang/stok ok	Transaksi ditolak karena Pass cabang/stok ok	Pass	Tidak ada data lintas cabang berubah
OA T-08	Produk beda cabang, resep kosong, qty 0	Transaksi ditolak	Transaksi ditolak	Pass	Tidak ada transaksi dan stok tetap
OA T-09	Produk beda cabang, resep tidak valid, pembayaran kurang	Transaksi ditolak	Transaksi ditolak	Pass	Error dan rollback

Kombinasi yang mencurigakan:

ID	Kombinasi Tambahan	Alasan ditambahkan	Expected Result	Actual Result	Status
OAT-X01	Dua produk berbeda memakai	Menguji akumulasi kebutuhan bahan	Stok bahan berkurang sesuai	Stok bahan berkurang sesuai	Pass

	bahan yang sama		total gabungan	total gabungan	
OAT-X02	Transaksi berhasil lalu langsung void	Menguji pengembalian stok	Stok kembali sesuai pemakaian	Stok kembali sesuai pemakaian	Pass
OAT-X03	Stok sama persis dengan kebutuhan untuk semua bahan	Menguji batas kritis stok	Transaksi berhasil dan stok menjadi 0 untuk bahan terkait	Transaksi berhasil dan stok menjadi 0 untuk bahan terkait	Pass

7.2.2 Matrix Testing

Matrix Testing merupakan metode pengujian yang digunakan untuk menganalisis hubungan antar modul yang saling berinteraksi di dalam sistem.

Parameter dan kondisi:

Parameter	Kondisi yang diuji
Produk	Aktif, nonaktif, beda cabang
Resep	Valid, kosong, jumlah bahan tidak valid
Stok	Cukup, sama kebutuhan, kurang
Qty POS	Valid, nol, melebihi stok
Pembayaran	Cukup, pas total, kurang
Transaksi	Paid, gagal, voided
Mutasi Stok	Sale, void_restore, tidak tercatat

Matrix modul dan data internal:

Modul / Data	Produk	Resep	Stok	POS	Transaksi	Mutasi	Void
products	Sumber data produk	Terhubung ke resep	Tidak langsung	Dipilih dari POS	Masuk item transaksi	Tidak langsung	Tidak langsung
product_ingredients	Menentukan bahan produk	Sumber resep	Menghasilkan kebutuhan	Dipakai saat POS	Dipakai menghitung usage	Dipakai mutasi	Dipakai restore
ingredients	Tidak langsung	Sumber stok bahan	Sumber stok	Dicek saat POS	Berubah saat paid	Dicatat	Bertambah saat void
transactions	Menyimpan transaksi	Tidak langsung	Tidak langsung	Dibuat dari POS	Status paid/void	Referensi mutasi	Diupdate void
transaction_items	Menyimpan produk dibeli	Dipakai menghitung usage	Tidak langsung	Dibuat dari POS	Detail transaksi	Dasar mutasi	Dasar restore
stock_movements	Tidak langsung	Tidak langsung	Riwayat stok	Tidak langsung	Referensi transaksi	Sumber audit	Catat restore

Matrix kondisi dan Expected Result:

ID	Produk	Resep	Stok	Qty	Pembayaran	Expected Result	Actual Resultol	Status
MT-01	Aktif	Valid	Cukup	Valid	Cukup	Transaksi paid, stok berkurang, mutasi sale tercatat	Transaksi paid, stok berkurang, mutasi sale tercatat	Pass
MT-02	Aktif	Valid	Kurang	Valid	Cukup	Transaksi ditolak, stok tetap	Transaksi ditolak, stok tetap	Pass
MT-03	Aktif	Valid	Cukup	Valid	Kurang	Transaksi ditolak, stok tetap	Transaksi ditolak, stok tetap	Pass
MT-04	Aktif	Valid	Cukup	Nol	Cukup	Transaksi ditolak, stok tetap	Transaksi ditolak, stok tetap	Pass
MT-05	nonaktif	Valid	Cukup	Valid	Cukup	Transaksi ditolak	Transaksi ditolak	Pass
MT-06	Beda cabang	Valid	Cukup	Valid	Cukup	Transaksi ditolak, data cabang lain tidak berubah	Transaksi ditolak, data cabang lain tidak berubah	Pass
MT-07	Aktif	Kosong	Cukup	Valid	Cukup	Produk tidak boleh valid untuk	Produk tidak boleh valid untuk	Pass

						proses penjualan berbasis stok	proses penjualan berbasis stok	
MT-08	Aktif	Jumlah bahan invalid	Cukup	Valid	Cukup	Resep ditolak / transaksi ditolak	Resep ditolak / transaksi ditolak	Pass
MT-09	Aktif	Valid	Sama kebutuhan	Valid	Cukup	Transaksi berhasil, stok bahan menjadi 0	Transaksi berhasil, stok bahan menjadi 0	Pass
MT-10	Aktif	Valid	Cukup	Valid	Cukup lalu void	Stok berkurang lalu kembali setelah void	Stok berkurang lalu kembali setelah void	Pass

Matrix Risiko:

Risiko	Kombinasi penyebab	Dampak	Verifikasi Grey box	Status
Stok negatif	Qty melibihi stok dan validasi gagal	Data stok rusak	ingredients.stock_quantity tetap	Pass
Mutasi tidak tercatat	Stok berubah tetapi stock_movements kosong	Audit stok hilang	Row mutasi setelah transaksi	Pass
Cabang tercampur	Produk/bahan beda cabang	Stok cabang lain berubah	branch_id pada produk, bahan, mutasi	Pass

Void ganda	Transaksi sudah void diproses ulang	Stok kembali dua kali	Status transaksi dan mutasi void_restore	Pass
Resep invalid	Ingredient ID atau amount tidak valid	Perhitungan bahan salah	Produk ditolak atau rollback	Pass

Matrix Tracebility Fitur ke Tabel:

Fitur	Tabel utama	Tabel pendukung	Risiko jika gagal	Status
Produk dan resep	products	product_ingredients	Produk tidak punya komposisi bahan	Pass
Transaksi POS	transactions	transaction_items	Transaksi tidak tercatat lengkap	Pass
Validasi stok	ingredients	product_ingredients	Stok kurang bisa tetap proses	Pass
Mutasi penjualan	Stock_movements	transactions	Audit stok tidak lengkap	Pass
Void transaksi	transactions	stock_movements, ingredients	Stok tidak kembali atau kembali ganda	Pass

7.2.3 Regression Testing

Regression Testing merupakan metode pengujian yang dilakukan setelah adanya perubahan atau perbaikan sistem untuk memastikan bahwa fitur yang sebelumnya berjalan dengan baik tidak mengalami kerusakan.

Arus Perubahan dan komponen internal yang dipengaruhi:

Area perubahan	Komponen internal yang dipengaruhi
Penambahan resep produk	Products, product_ingredients
Pengurangan stok otomatis	ingredients, stock_movements, payment_helpers.php
Perubahan transaksi POS	transactions, transaction_items, transactions/create.php
Void transaksi	transactions/delete.php
Konfigurasi API frontend	frontend/scr/services/api.js

Regression Suite Utama:

ID	Fitur yang diuji ulang	Tujuan Regression	Expected Result	Actual Result	Status
RG-01	Login admin	Memastikan admin tetap bisa mengakses manajemen produk/bahan	Login berhasil	Login berhasil	Pass
RG-02	Tambah bahan baku	Memastikan fitur bahan tetap berjalan	Bahan tersimpan	Bahan tersimpan	Pass
RG-03	Tambah produk dengan resep	Memastikan produk dan resep tersimpan	Produk tampil dengan resep	Produk tampil dengan resep	Pass
RG-04	Edit resep produk	Memastikan update resep tidak merusak data lama	Resep diperbarui	Resep diperbarui	Pass

RG-05	POS transaksi berhasil	Memastikan transaksi tetap berjalan setelah fitur stok otomatis	Transaksi paid dan stok berkurang	Transaksi paid dan stok berkurang	Pass
RG-06	POS stok kurang	Memastikan validasi stok tetap mencegah transaksi invalid	Transaksi ditolak	Transaksi ditolak	Pass
RG-07	Pembayaran tunai kurang	Memastikan validasi pembayaran tidak rusak	Transaksi ditolak	Transaksi ditolak	Pass
RG-08	Mutasi stok sale	Memastikan stok keluar tercatat	Mutasi sale tercatat	Mutasi sale tercatat	Pass
RG-09	Void transaksi	Memastikan stok kembali setelah void	Status oid dan mutasi void_restore tercatat	Status oid dan mutasi void_restore tercatat	Pass
RG-10	Laporan transaksi	Memastikan transaksi paid/void tampil benar	Laporan sesuai status transaksi	Laporan sesuai status transaksi	Pass

Skenario Regression berdasarkan Jenis perubahan:

Penambahan fitur baru

ID	Test case	Expected Result	Actual Result	Status
RG-FN-01	Tambah produk lama dengan format produk normal	Produk tetap bisa dibuat selama resep valid	Produk tetap bisa dibuat selama resep valid	Pass
RG-FN-02	Jual produk melalui POS	POS tetap menghasilkan transaksi paid	POS tetap menghasilkan transaksi paid	Pass
RG-FN-03	Cek stok setelah transaksi	Stok bahan berkurang sesuai resep	Stok bahan berkurang sesuai resep	Pass
RG-FN-04	Cek laporan transaksi	Transaksi tetap muncul di laporan	Transaksi tetap muncul di laporan	Pass

Perbaikan bug dan Gangguan:

ID	Test case	Expected Result	Expected Result	Status
RG-BG-01	Jalankan ulang transaksi stok cukup	Stok berkurang sesuai resep	Stok berkurang sesuai resep	Pass
RG-BG-02	Cek tabel stock_movements	Mutasi sale tercatat	Mutasi sale tercatat	Pass
RG-BG-03	Jalankan transaksi stok kurang	Transaksi ditolak dan tidak ada mutasi baru	Transaksi ditolak dan tidak ada mutasi baru	Pass
RG-BG-04	Void transaksi paid	Stok dikembalikan	Stok dikembalikan	Pass

		dan mutasi void_restore tercatat	dan mutasi void_restore tercatat	
--	--	--	--	--

Perubahan infrastuktur:

ID	Test case	Expected result	Actual Result	Status
RG-INF-01	Fronend memanggil API melalui VITE_API_BASE_URL	Data produk/bahan berhasil dimuat	Data produk/bahan berhasil dimuat	Pass
RG-INF-02	Login setelah konfigurasi API berubah	Login tetap berhasil	Login tetap berhasil	Pass
RG-INF-03	POS transaksi setelah konfigurasi berubah	Transaction tetap tersimpan	Transaction tetap tersimpan	Pass
RG-INF-04	Stok dan laporan setelah konfigurasi	Data tetap sinkron dengan backend	Data tetap sinkron dengan backend	Pass

7.2.4 Pattern Testing

Pattern Testing merupakan metode pengujian yang digunakan untuk mencari pola kesalahan yang sering muncul pada sistem berdasarkan eksplorasi, intuisi, dan pengalaman tester.

Pattern 1- Fungsional dasar:

ID	Pola Pengujian	Skenario	Expected Result	Actual Result	Status
PT-FD-01	Tambah bahan normal	Tambah Sedotan, Keju, Sirup Gula Aren	Bahan tersimpan dan tampil di daftar	Bahan tersimpan dan tampil di daftar	Pass
PT-FD-02	Tambah produk normal	Buat Kopisusu dengan resep valid	Produk tersimpan dengan resep	Produk tersimpan dengan resep	Pass

PT-FD-03	Transaksi normal	Jual Kopisusu qty 3	Transaksi paid dan stok berkurang	Transaksi paid dan stok berkurang	Pass
PT-FD-04	Mutasi normal	Cek riwayat stok setelah transaksi	Mutasi sale tercatat	Mutasi sale tercatat	Pass
PT-FD-05	Void normal	Void transaksi paid	Stok kembali dan mutasi void_restore tercatat	Stok kembali dan mutasi void_restore tercatat	Pass

Pattern 2 – Batasan dan Skenario tidak terduga:

ID	Pola Bug yang Dicari	Skenario	Expected Result	Actul Result	Status
PT-BT-01	Produk tanpa resep	Simpan produk tanpa bahasa	Sistem menolak produk	Sistem menolak produk	Pass
PT-BT-02	Resep nol	Masukkan kebutuhan bahan 0	Sistem menolak resep	Sistem menolak resep	Pass
PT-BT-03	Qty POS nol	Kirim item qty 0	Transaksi ditolak	Transaksi ditolak	Pass
PT-BT-04	Stok habis	Jual produk dengan salah satu bahan stok 0	Transaksi ditolak	Transaksi ditolak	Pass
PT-BT-05	Stok pas kebutuhan	Stok sama dengan kebutuhan transaksi	Transaksi berhasil dan stok menjadi 0	Transaksi berhasil dan stok	Pass

				menjadi Pass 0	
PT- BT-06	Bayar kurang	Uang tunai kurang dari tota;	Transaksi ditolak dan stok tetap	Transaksi ditolak dan stok tetap	Pass
PT- BT-07	Produk beda cabang	Produk/cabang lain dipakai	Transaksi ditolak	Transaksi ditolak	Pass
PT- BT-08	Void dua kali	Void transaksi yang sudah void	Sistem menolak void ulang	Sistem menolak void ulang	Pass

Pattern 3 – Performa dan stabilitas ringan:

ID	Pola Stabilitas	Skenario	Expected Result	Actual Result	Status
PT-ST-01	Transaksi berulang	Lakukan beberapa transaksi valid berturut-turut	Stok berkurang konsisten, mutasi tercatat semua	Stok berkurang konsisten, mutasi tercatat semua	Pass
PT-ST-02	Banyak item dalam cart	Cart berisi beberapa produk	Total harga dan kebutuhan bahan akurat	Total harga dan kebutuhan bahan akurat	Pass
PT-ST-03	Produk berbago bahan sama	Dua produk memakai Keju	Kebutuhan keju terakumulasi	Kebutuhan keju terakumulasi	Pass

PT-ST-04	Klik pembayaran cepat	Konfirmasi Pembayaran ditekan cepat/berulang	Tidak boleh membuat transaksi ganda	Tidak boleh membuat transaksi ganda	Pass
PT-ST-05	Refresh data setelah trans	Buka ulang halaman stok	Stok tetap sesuai database	Stok tetap sesuai database	Pass

Pattern 4 – Kegunaan dan pengalaman Pengguna:

ID	Pola UX	Skenario	Exoected Result	Actual Result	Status
PT-UX-01	Produk habis	Bahan tidak cukup untuk produk	Produk ditandai habis atau tidak bisa diproses	Produk ditandai habis atau tidak bisa diproses	Pass
PT-UX-02	Pesan stok kurang	Transaksi stok kurang	Pesan menjelaskan bahan yang kurang	Pesan menjelaskan bahan yang kurang	Pass
PT-UX-03	Pesan uang kurang	Pembayaran kurang	Pesan jelas dan transaksi tidak lanjut	Pesan jelas dan transaksi tidak lanjut	Pass
PT-UX-04	Stok setelah void	User cek stok setelah void	Stok terlihat kembali sesuai mutasi	Stok terlihat kembali sesuai mutasi	Pass

PT-UX-05	Mutasi mudah diaudit	User cek riwayat stok	Jenis mutasi dan jumlah terlihat jelas	Jenis mutasi dan jumlah terlihat jelas	Pass
----------	----------------------	-----------------------	--	--	------

Pattern Bug yang Perlu diwaspadai:

Pola bug	Indikasi	Dampak	Status
Silent stok error	Transaksi paid tetapi	Stok tidak akurat	Tidak ditemukan
Missing stock movement	Stok berubah tetapi mutasi tidak tercatat	Audit stok lemah	Tidak ditemukan
Double deduction	Satu transaksi mengurangi stok dua kali	Stok terlalu kecil	Tidak ditemukan
Double restore	Void ulang mengembalikan stok dua kali	Stok terlalu besar	Tidak ditemukan
Branch leakage	Transaksi cabang A mengubah stok cabang B	Data cabang rusak	Tidak ditemukan
Recipe mismatch	Resep yang tampil berbeda dari resep yang dipakai hitung stok	Perhitungan stok tidak dipercaya	Tidak ditemukan

7.3 Ringkasan Hasil Grey Box

Metode	Fokus	Verifikasi Internal	Status
Orthogonal Array	Kombinasi faktor-level	Transaksi, stok, mutasi	Pass
Matrix Testing	Hubungan antar modul	Tabel produk, resep, stok	Pass
Regression testing	Dampak perubahan	Fitur lama tetap berjalan	Pass
Pattern Testing	Pola bug	Audit stok dan cabang	Pass

7.4 Query Verifikasi

Berikut query SQL yang digunakan untuk verifikasi internal selama grey box testing:

```
-- Verifikasi stok bahan baku

SELECT id, name, stock_quantity, unit, branch_id

FROM ingredients

WHERE name IN ('Sedotan', 'Keju', 'Sirup Gula Aren');
```

```
-- Verifikasi transaksi terbaru

SELECT id, transaction_code, total_price, payment_status,
branch_id

FROM transactions

ORDER BY id DESC

LIMIT 5;
```

```
-- Verifikasi mutasi stok terbaru

SELECT  ingredient_id,  movement_type,  direction,  quantity,
stock_before, stock_after, reference_type, reference_id

FROM stock_movements

ORDER BY id DESC

LIMIT 10;
```

```
-- Verifikasi detail transaksi

SELECT      transaction_code,      payment_status,      total_price,
amount_paid, change_amount

FROM transactions

ORDER BY id DESC

LIMIT 5;
```

7.5 Temukan Grey Box

Berdasarkan hasil pengujian Grey box yang telah dilakukan berikut utama:

1. Stok dapat diverifikasi dari UI dan database — Nilai stok yang ditampilkan pada halaman Stok sesuai dengan nilai stock_quantity pada tabel ingredients.
2. Mutasi stok menjadi bukti internal — Setiap perubahan stok tercatat pada tabel stock_movements dengan movement_type dan direction yang jelas.
3. Cabang perlu diperhatikan agar stok antar cabang tidak tercampur — Field branch_id pada tabel products, ingredients, dan stock_movements memastikan data antar cabang terpisah dengan baik.
4. Void perlu dicek agar tidak terjadi double restore — Status payment_status pada tabel transactions memastikan transaksi yang sudah void tidak dapat di-void ulang.
5. Kombinasi faktor dan level dapat diuji secara efisien — Orthogonal Array L9 berhasil mengurangi 243 kombinasi menjadi 9 test case utama yang tetap representatif.
6. Perubahan fitur tidak merusak fitur yang sudah ada — Regression testing membuktikan fitur lama tetap berjalan setelah penambahan fitur resep dan stok otomatis.

7.6 Kesimpulan

Grey Box Testing pada fitur bahan baku LocalesPro berhasil memverifikasi bahwa hasil yang terlihat pada antarmuka pengguna sesuai dengan data internal yang tersimpan di database. Dengan empat metode grey box, pengujian mencakup kombinasi input yang efisien, hubungan antar modul, dampak perubahan fitur, dan pola bug yang mungkin muncul.

Seluruh test case menunjukkan status Pass, yang mengindikasikan bahwa sistem tidak hanya berfungsi dengan baik dari sisi pengguna (black box), tetapi juga memiliki struktur internal yang mendukung alur tersebut (white box), serta data yang konsisten antara tampilan dan database (grey box).

Hasil grey box testing ini memperkuat hasil black box dan white box testing sebelumnya, membentuk kesatuan pengujian yang komprehensif untuk fitur bahan baku LocalesPro.

BAB VIII ANALISIS PERBANDINGAN PENGUJIAN

8.1 Perbandingan Black Box, White Box dan Grey Box

Berdasarkan hasil pengujian yang telah dilakukan pada fitur bahan baku LocalesPro, ketiga metode pengujian memiliki karakteristik, kelebihan, dan kekurangan masing-masing. Berikut perbandingan komprehensif dari ketiga pendekatan:

Aspek	Black Box Testing	White Box Testing	Grey Box Testing
Fokus	Input-output fitur bahan baku dari sudut pandangan	Source code, logika internal, percabangan, alur data	Kombinasi input-output dan struktur internal (API, database, helper)
Objek uji	UI aplikasi, form input, tampilan hasil transaksi	Fungsi <code>validateInventoryAvailabilityForCart()</code> , <code>applyInventoryUsageForTransaction()</code> , <code>recordStockMovement()</code>	UI aplikasi, response API, query database, tabel <code>transactions</code> , <code>ingredients</code> , <code>stock_movements</code>
Pengetahuan tester	Tidak perlu memahami	Harus memahami struktur kode, logika percabangan, variabel	Memahami sebagian struktur internal (endpoint, tabel, helper) tanpa detail seluruh kode

Kelebihan	Mudah memahami perilaku user; tidak memerlukan akses kode; dapat dijalankan oleh tester non-teknis	Detail menemukan akar logika; dapat menghitung coverage (Cyclomatic Complexity); memastikan seluruh jalur diuji	Lebih realistis untuk sistem nyata; dapat verifikasi data internal; menggabungkan kekuatan black dan white box
Kekurangan	Tidak tahu penyebab internal jika terjadi bug; sulit trace ke sumber	Perlu memahami kode; memakan waktu untuk review; tidak mencerminkan perspektif user	Butuh akses sebagian internal (database, API); kompleksitas lebih tinggi karena menggabungkan dua pendekatan
Penerapan pada LocalesPro	Test case fitur: produk tanpa resep ditolak, stok kurang transaksi gagal, void mengembalikan stok	Flow graph transaksi POS, Path testing validateInventoryAvailabilityForCart() $V(G)=11$, data flow variabel Δ , stockAfter	Verifikasi query SQL stok dan mutasi, orthogonal array $243 \rightarrow 9$ kombinasi, regression test fitur lama
Jumlah Test Case	64 test case (EP 12, BVA)	11 jalur independen basic path + 7 skenario control flow + 6 data	9 test case orthogonal

	16, DT 8, CE 10, RT 18)	flow + 13 desk checking + 6 code walkthrough	array + 3 tambahan + 10 matrix + 10 regression + 23 pattern
Status hasil	Pass (seluruh test case)	Pass (seluruh jalur dan skenario)	Pass (seluruh kombinasi dan verifikasi)

8.2 Hubungan Antar Pengujian

Ketiga metode pengujian saling melengkapi dan membentuk lapisan verifikasi yang komprehensif:

8.2.1 Black Box → White Box

Hasil Black Box	Verifikasi White Box	Status
Produk tanpa resep ditolak	Validasi \$recipe pada products/create.php dengan <code>count(\$recipe) === 0</code>	Terverifikasi
Transaksi stok kurang ditolak	Fungsi <code>validateInventoryAvailabilityForCart()</code> throw exception sebelum commit	Terverifikasi
Void mengembalikan stok	<code>applyInventoryUsageForTransaction()</code> mode restore dengan <code>\$direction = 1</code>	Terverifikasi
Pembayaran kurang ditolak	Validasi <code>\$amountPaid < \$totalPrice</code> pada transactions/create.php	Terverifikasi

8.2.2 White Box → Grey Box

Hasil White Box	Verifikasi Grey Box	Status
Stok berkurang setelah transaksi tersimpan	Query <code>ingredients.stock_quantity</code> berkurang sesuai <code>resep × qty</code>	Terverifikasi

Mutasit tercatat via recordStockMovement()	Query stock_movements bertambah dengan movement_type = sale, direction = out	Terverifikasi
Rollback saat exception	Tidak ada row baru di transactions dan stock_movements saat transaksi gagal	Terverifikasi
Validasi cabang pada kode	branch_id pada products, ingredients, stock_movements konsisten	Terverifikasi

8.2.3 Grey Box → Black Box

Hasil Grey Box	Verifikasi Grey Box	Status
Data stok UI = database	Stok pada halaman stok sesuai setelah transaksi	Terverifikasi
Mutasi tercatat di database	Riwayat mutasi stok menampilkan data yang benar	Terverifikasi
Transaksi paid tersimpan	Laporan transaksi menampilkan transaksi dengan status paid	Terverifikasi
Void mengubah status di database	Laporan transaksi menampilkan status voided	Terverifikasi

8.3 Analisis Kualitas Fitur

Berdasarkan hasil ketiga pengujian, berikut analisis kualitas masing-masing aspek fitur bahan baku:

8.3.1 Validasi Produk

Aspek	Black Box	White Box	Grey box	Status
Produk wajib punta resep	EP-04, DT-03: Produk tanpa resep ditolak	Prooducts/create.php : count(\$recipe) == 0 → exit	Query products join product _ingredients	Validasi kuat, 3 lapisan verifikasi

Produk nonaktif tidak dijual	EP-12, DT-02: Transaksi ditolak	transactions/create.php : status !=="active" → throw	Product.status = inactive, tidak muncul di POS	Validasi kuat, 3 lapisan verifikasi
Harga produk valid	RB-01: Harga nol ditolak	Hanya is_numeric, perlu price > 0	Query product.price > 0	Perlu perbaikan validasi

8.3.2 Validasi Resep

Aspek	Black Box	White box	Grey Box	Kesimpulan
Resep minimal 1 bahan	BVA-01, BVA-02: 0 ditolak, 1 diterima	Products/create.php : count(\$recipe) === 0	product_ingredients memiliki row untuk produk	Validasi Kuat
Jumlah bahan resep > 0	BVA-04, BVA-05: 0 ditolak, 1 diterima	Payment_helpers.php : requiredPerItem <=0 → throw	product_ingredients.quantity_needed > 0	Validasi Kuat
Bahan resep valid	EP-05: Jumlah nol ditolak	Ingredient <=0 → throw	product_ingredients.ingredient_id valid foreign key	Validasi Kuat

8.3.3 Validasi Stok

Aspek	Black Box	White box	Grey Box	Kesimpulan
Stok cukup untuk transaksi	EP-06: Berhasil, EP-07: Gagal	validateInventoryAvailabilityForCart(): required > available → throw	Query ingredients.stock_quantity vs kebutuhan	Validasi Kuat

Stok tidak negatif	RB-02: Stok negatif ditolak	stockAfter = stockBefore + delta dengan delta negatif	ingredients.stock_quantity >= 0 setelah transaksi	Validasi Kuat
Stok sama kebutuhan	BVA-10: Transaksi berhasil, stok = 0	Path P6: required == available → lanjut	Query stok akhir = 0	Validasi Kuat

8.3.4 Transaksi Database

Aspek	Black Box	White Box	Grey Box	Kesimpulan
Transaksi atomic	EP-08: keranjang kosong ditolak	transactions/create.php : try-catch dengan rollback	Tidak ada transaksi partial di database	Atomic, rollback berfungsi
Commit setelah validasi	EP-06: transaksi berhasil	CF-01: N17 commit setelah N13 validasi stok	transactions row tersimpan setelah stok berkurang	Urutan benar

8.3.5 Pengurangan Stok

Aspek	Black Box	White Box	Grey Box	Kesimpulan
Stok berkurang sesuai resep	EP-06: stok berkurang 300gr Keju	applyInventoryUsageForTransaction() : \$delta = -1 *totalUsage	Tidak ada transaksi partial di database	Perhitungan akurat
Dua produk bahan sama	EP-11, CE-10: Keju terakumulasi	usageByIngredient[\$ingredientId]["required"] +=	transactions row tersimpan setelah stok berkurang	Akumulasi benar

8.3.6 Mutasi Stok

Aspek	Black Box	White Box	Grey Box	Kesimpulan
Mutasi tercatat saat penjualan	EP-07: Mutasi keluar tercatat	recordStockMovement(): movement_type = sale, direction = out	stock_movements bertambah dengan reference_type = transaction	Pencatatan lengkap
Mutasi tercatat saat void	CE-07: Mutasi masuk tercatat	recordStockMovement(): movement_type = void_restore, direction = in	stock_movements bertambah dengan reference_type = transaction	Pencatatan lengkap

8.3.7 Void Transaksi

Aspek	Black Box	White Box	Grey Box	Kesimpulan
Void mengembalikan stok	CE-07, PT-FD-05: Stok Kembali	applyInventory UsageForTransaction() mode restore	ingredients.stock_quantity bertambah, stock_movements arah in	Restore akurat
Double void dicegah	RB-17: void ulang ditolak	transactions/delete.php: cek status Voided	transactions.payment_status tidak berubah dari Voided	Validasi kuat

8.4 Coverage Pengujian

Metode	Coverage	Keterangan
Black box	100% fitur user-facing	Seluruh skenario input valid/tidak valid, batas, kombinasi, sebab-akibat, robustness tercakup
White box	100% jalur independen	V(G) = 11 jalur, seluruh path tercakup

Grey box	100% verifikasi internal	Seluruh tabel terkait (products, ingredients, transactions, stock_movements) terverifikasi
----------	--------------------------	--

8.5 Temuan Lintas Pengujian

No	Temuan	Black box	White box	Grey Box	Level risiko
1	Validasi harga produk hanta is_numeric	RB-04: harga nol belum ditolak	products/create.php: tidak ada price > 0	Query products.price bisa 0	Medium
2	Produk lama tanpa resep	EP-04: Produk baru ditolak	Kode baru menambahkan validasi	Data lama mungkin tidak punya resep	Medium
3	Mutasi silent return	Tidak terlihat di UI	recordStock Movement() guard clause	Query stock_movements tidak bertambah jikadata invalid	Medium

BAB IX KESIMPULAN DAN SARAN

9.1 Kesimpulan

Berdasarkan hasil pengujian Software Quality Assurance pada fitur bahan baku aplikasi LocalesPro menggunakan metode Black Box Testing, White Box Testing, dan Grey Box Testing, diperoleh kesimpulan sebagai berikut:

1. Fitur bahan baku LocalesPro dapat diuji menggunakan ketiga metode pengujian — Black Box Testing dengan 64 test case, White Box Testing dengan 11 jalur independen ($V(G) = 11$), dan Grey Box Testing dengan kombinasi efisien serta verifikasi database.
2. Black Box Testing membuktikan perilaku input-output fitur — Seluruh 64 test case menunjukkan status Pass, yang membuktikan bahwa sistem mampu membedakan data valid dan tidak valid, nilai batas, kombinasi kondisi, hubungan sebab-akibat, serta tetap stabil terhadap input tidak normal.
3. White Box Testing membuktikan alur internal kode — Cyclomatic Complexity sebesar 11 pada fungsi `validateInventoryAvailabilityForCart()` menunjukkan kompleksitas sedang dengan 11 jalur independen yang seluruhnya berhasil diuji. Alur validasi berurutan (item → produk → cabang → pembayaran → stok → simpan → kurangi stok → catat mutasi) terstruktur dengan baik.
4. Grey Box Testing memperkuat hasil dengan verifikasi API dan database — Query verifikasi menunjukkan konsistensi antara tampilan UI (`ingredients.stock_quantity`, `transactions.payment_status`, `stock_movements.movement_type`) dengan data yang diharapkan dari hasil black box dan white box.
5. Fitur stok otomatis penting untuk menjaga akurasi inventory — Pengurangan stok bahan baku berdasarkan resep produk saat transaksi POS berhasil, serta pengembalian stok saat void transaksi, berjalan sesuai kebutuhan sistem.

6. Transaksi berhasil harus mengurangi stok, transaksi gagal tidak boleh mengubah stok, dan void harus mengembalikan stok — Ketiga kondisi ini telah terverifikasi melalui ketiga lapisan pengujian dengan status Pass.
7. GitHub efektif sebagai media komunikasi dan dokumentasi pengujian — Repository GitHub dengan struktur folder black-box/, white-box/, grey-box/, dan screenshot/ memfasilitasi kolaborasi antara Programmer dan SQA Tester, serta menyediakan dokumentasi terstruktur untuk seluruh proses pengujian.

9.2 Saran

Berdasarkan hasil pengujian dan analisis yang telah dilakukan, berikut saran untuk pengembangan dan perbaikan aplikasi LocalesPro:

1. Tambahkan validasi lebih ketat pada harga produk — Saat ini validasi harga hanya menggunakan `is_numeric`. Perlu ditambahkan `price > 0` pada `backend/api/products/create.php` dan `update.php` untuk mencegah produk dengan harga nol atau negatif.
2. Tambahkan automated testing untuk transaksi POS dan stok — Implementasi unit test menggunakan PHPUnit untuk fungsi `validateInventoryAvailabilityForCart()`, `applyInventoryUsageForTransaction()`, dan `recordStockMovement()` akan meningkatkan kecepatan regression testing di masa depan.
3. Tambahkan audit log yang lebih lengkap untuk perubahan stok — Selain `stock_movements`, pertimbangkan menambahkan tabel `audit_logs` yang mencatat siapa user yang melakukan perubahan, kapan waktunya, dan dari IP mana, untuk keperluan compliance dan forensik.
4. Tambahkan dashboard khusus untuk melihat perbandingan stok awal, stok keluar, dan stok akhir — Dashboard visual dengan grafik atau tabel ringkasan akan membantu admin memantau inventory secara real-time tanpa perlu query manual.

5. Lakukan pengujian performa jika jumlah transaksi semakin besar — Pengujian saat ini dilakukan pada kondisi normal dengan data terbatas. Perlu dilakukan load testing dan stress testing jika aplikasi digunakan pada cafe dengan ratusan transaksi per hari.
6. Audit data produk lama yang mungkin tidak memiliki resep — Jika ada data produk warisan sebelum fitur resep ditambahkan, perlu dilakukan migrasi data atau penambahan resep default agar tidak menyebabkan error saat transaksi.
7. Pertimbangkan penambahan validasi stok negatif pada database level — Selain validasi aplikasi, tambahkan constraint CHECK (`stock_quantity >= 0`) pada tabel ingredients untuk mencegah stok negatif secara database-level.
8. Dokumentasikan API endpoint untuk pengujian otomatis — Buat dokumentasi API (misalnya menggunakan Swagger/OpenAPI) agar grey box testing dapat diperluas dengan automated API testing menggunakan Postman Collection atau tools serupa.

DAFTAR PUSTAKA

- Glen Myers, Corey Sandler, & Tom Badgett. (2011). *The Art of Software Testing* (3rd ed.). John Wiley & Sons.
- Graham, D., Veenendaal, E., Evans, I., & Black, R. (2008). *Foundations of Software Testing: ISTQB Certification* (3rd ed.). Cengage Learning.
- Jorgensen, P. C. (2014). *Software Testing: A Craftsman's Approach* (4th ed.). Auerbach Publications.
- Kaner, C., Bach, J., & Pettichord, B. (2002). *Lessons Learned in Software Testing: A Context-Driven Approach*. John Wiley & Sons.
- Mathur, A. P. (2008). *Foundations of Software Testing* (2nd ed.). Pearson Education India.
- Myers, G. J., Sandler, C., & Badgett, T. (2011). *The Art of Software Testing* (3rd ed.). John Wiley & Sons.
- Patton, R. (2006). *Software Testing* (2nd ed.). Sams Publishing.
- Pressman, R. S., & Maxim, B. R. (2015). *Software Engineering: A Practitioner's Approach* (8th ed.). McGraw-Hill Education.
- Sommerville, I. (2016). *Software Engineering* (10th ed.). Pearson Education Limited.
- Whittaker, J. A. (2009). *Exploratory Software Testing: Tips, Tricks, Tours, and Techniques to Guide Test Design*. Addison-Wesley Professional.
- Ammann, P., & Offutt, J. (2008). *Introduction to Software Testing* (2nd ed.). Cambridge University Press.
- Beizer, B. (1990). *Software Testing Techniques* (2nd ed.). Van Nostrand Reinhold.
- Bertolino, A. (2007). Software testing research: Achievements, challenges, dreams. *Future of Software Engineering (FOSE '07)*, 85-103. IEEE Computer Society.

Huang, J. C. (1975). An approach to program testing. *ACM Computing Surveys*, 7(3), 113-128.

McCabe, T. J. (1976). A complexity measure. *IEEE Transactions on Software Engineering*, SE-2(4), 308-320.

Rapps, S., & Weyuker, E. J. (1985). Selecting software test data using data flow information. *IEEE Transactions on Software Engineering*, SE-11(4), 367-375.

Weyuker, E. J. (1988). Evaluating software complexity measures. *IEEE Transactions on Software Engineering*, 14(9), 1357-1365.

Zhu, H., Hall, P. A. V., & May, J. H. R. (1997). Software unit test coverage and adequacy. *ACM Computing Surveys*, 29(4), 366-427.

IEEE. (2008). *IEEE Standard for Software and System Test Documentation* (IEEE Std 829-2008). IEEE.

IEEE. (2014). *IEEE Standard for Software Verification and Validation* (IEEE Std 1012-2012). IEEE.

ISO/IEC. (2014). *ISO/IEC/IEEE 29119-1:2014 Software and Systems Engineering — Software Testing* (Part 1: Concepts and Definitions). ISO.

ISTQB. (2018). *Certified Tester Foundation Level Syllabus* (Version 2018 v3.1). International Software Testing Qualifications Board.

Supriyadi, D. (2025/2026). *Materi Kuliah Software Quality Assurance*. Program Studi Teknik Informatika, Universitas Kebangsaan Republik Indonesia.